JEE

JSP

Introducción
Expresiones y Directivas
Procesamientos de datos en peticiones HTTP
Parámetros en formularios

Parte 1

INTRODUCCIÓN

- Una página JSP suele ser normalmente una página HTML con cierto contenido Java dinámico en su interior que utiliza unas etiquetas que un compilador JSP entenderá.

- En el caso de Tomcat este compilador se llama Jasper.

INTRODUCCIÓN

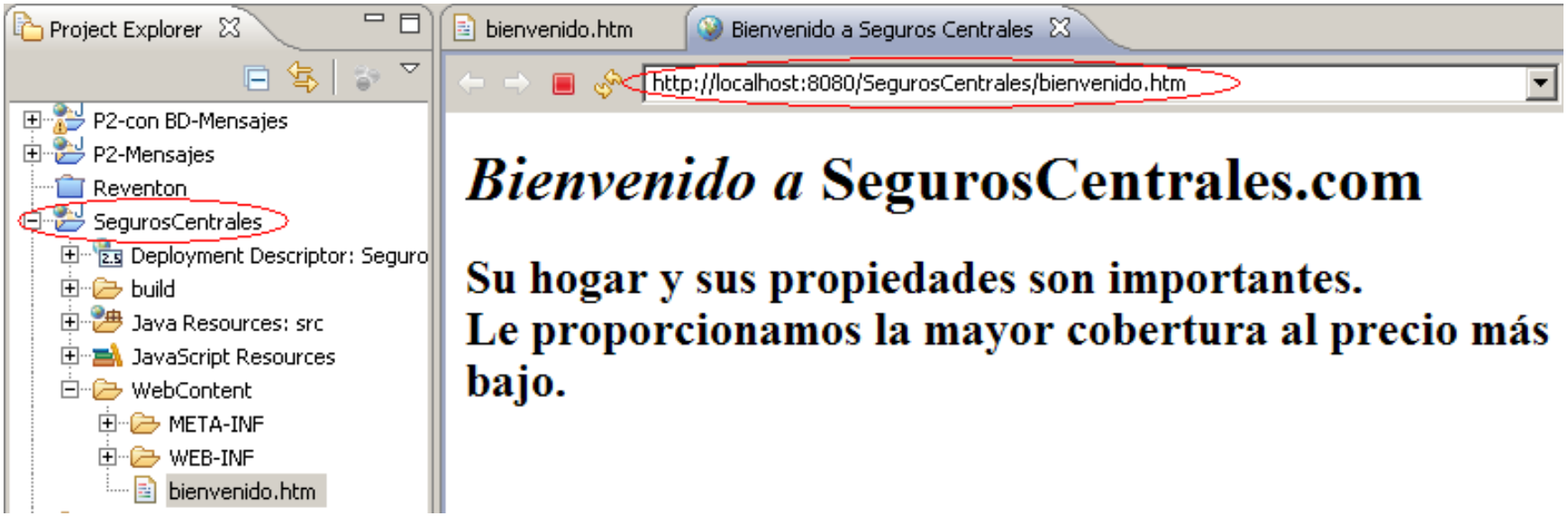
- Comenzamos viendo como transformar una página estática HTML en una JSP.

- Partimos de la siguiente página: bienvenido.htm

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
   "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
  <head>
    <title>Bienvenido a Seguros Centrales</title>
  </head>
  <body>
  <h1><em>Bienvenido a</em> SegurosCentrales.com</h1>
    <h2>
       Su hogar y sus propiedades son importantes.
    <br/>
       Le proporcionamos la mayor cobertura al precio más bajo.
    </h2>
  </body>
</html>
```

INTRODUCCIÓN

➢La página anterior podría formar parte un proyecto web dinámico en Eclipse (por supuesto, funcionaría en un proyecto estático):



INTRODUCCIÓN

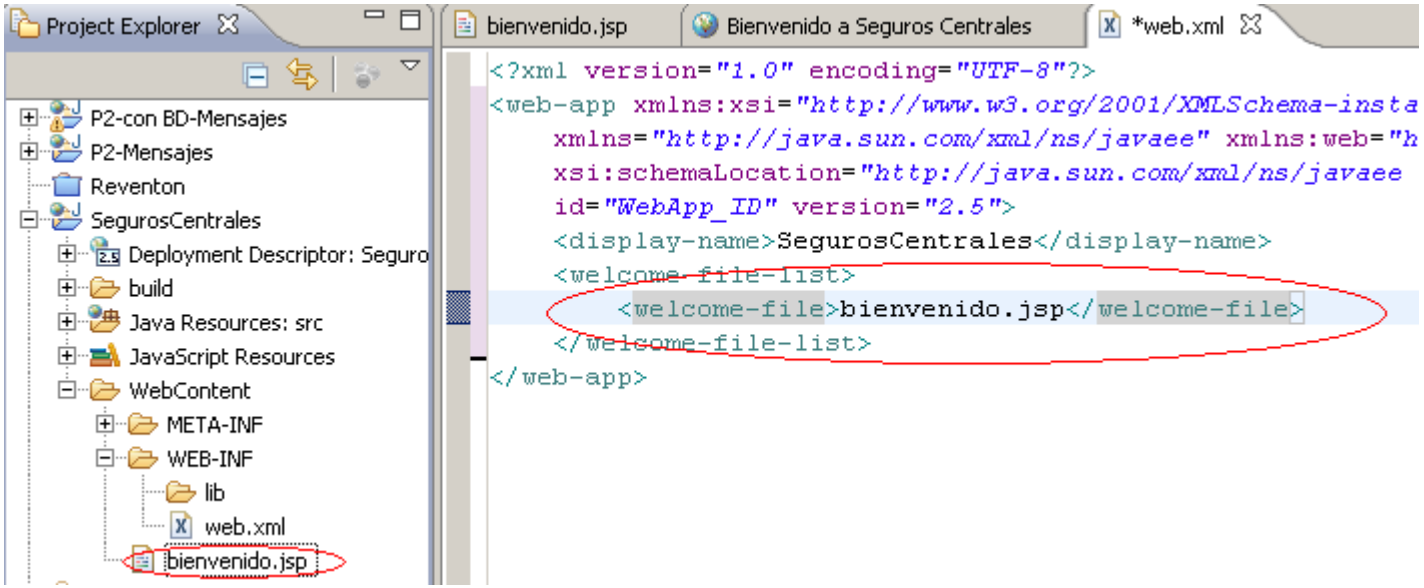
- Para que aparezca la página HTML al escribir el URL http://localhost:8080/SegurosCentrales tenemos que indicarlo en el descriptor de despliegue web.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
id="WebApp_ID" version="2.5">
<display-name>SegurosCentrales</display-name>
<welcome-file-list>
<welcome-file>bienvenido.htm</welcome-file>
</welcome-file-list>
</web-app>
```

INTRODUCCIÓN

- Ahora vamos a convertir la página bienvenido.htm en bienvenido.jsp.

- Para ello, sólo hemos de cambiar la extensión al fichero e indicar este cambio en el descriptor web.xml:



INTRODUCCIÓN

- Lógicamente no veremos diferencia alguna en los contenidos que visualiza la página, dado que no hemos escritor código dinámico.

- No obstante sí se comprobará que tarda algo más en aparecer que la página HTML original. Esto es debido a que la JSP primero se tiene que convertir en un servlet, el cual generará el contenido estático que es enviado al cliente (navegador).

COMO ESCRIBIR EXPRESIONES JSP

- Una expresión contiene cierto código Java que indica un valor que ha de ser escrito en la respuesta HTTP.

- Este valor puede provenir de cualquier cosa que pueda convertirse en un objeto String de Java.

- Lo único que no podemos poner dentro de una expresión es un método que devuelva un valor vacío y por tanto no pueda ser convertido en objeto String.

COMO ESCRIBIR EXPRESIONES JSP

- Las expresiones JSP tienen la sintaxis siguiente:

```
<%=valor %>
```

- Así, dentro de la etiqueta debemos indicar un valor o llamar a un método que devuelva un valor:

```
<%= new java.util.Date() %>
```

- Notemos como no hay que finalizar con un ';' como ocurriría en cualquier otro código fuente Java.

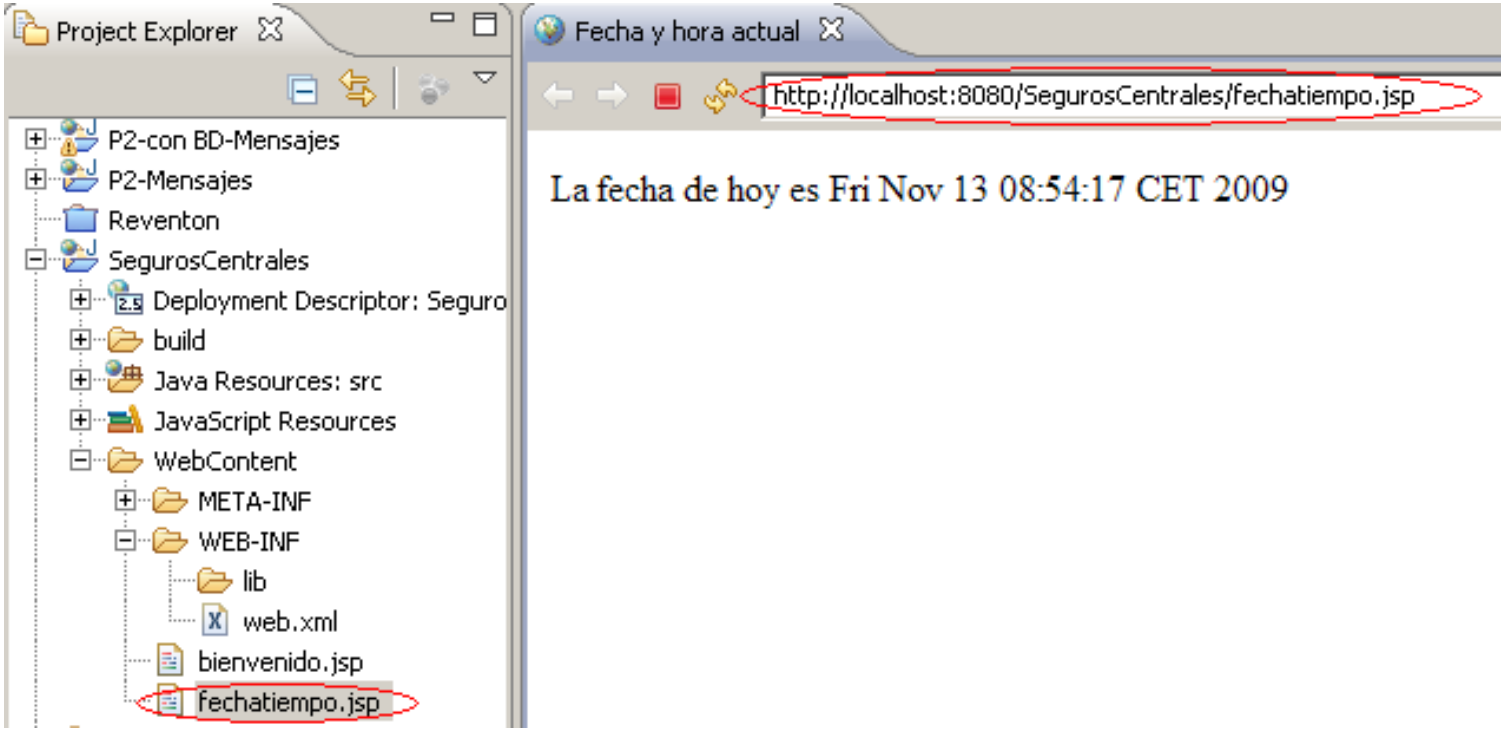
COMO ESCRIBIR EXPRESIONES JSP

- El siguiente código (fechatiempo.jsp) muestra una página JSP completa que genera una página XHTML válida e incluye la expresión JSP con la que se muestra una fecha y una hora:

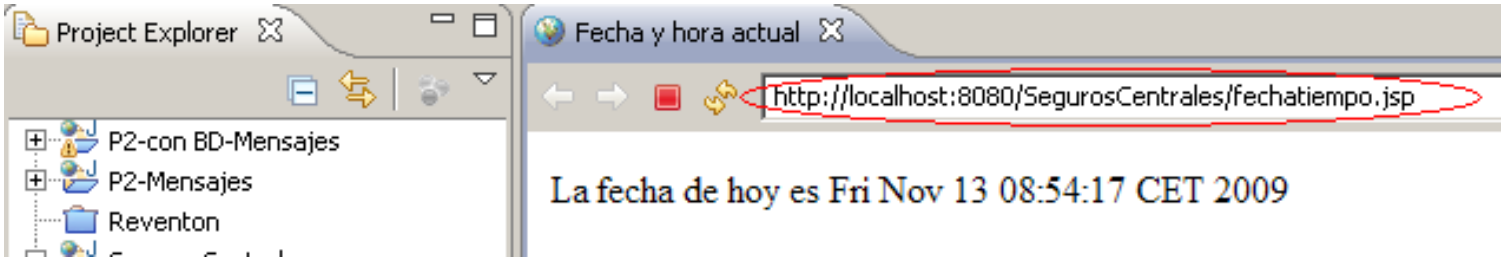
```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC
 "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <title>Fecha y hora actual</title>
 </head>
 <body>
 <!-- Fichero: fechatiempo.jsp -->
  <p>
  La fecha de hoy es
  <%= new java.util.Date() %>
  </p>
 </body>
</html>
```

COMO ESCRIBIR EXPRESIONES JSP

- Si hacemos http://localhost:8080/SegurosCentrales/fechatiempo.jsp



COMO ESCRIBIR EXPRESIONES JSP



- Podemos ver que la fecha se muestra de forma dinámica, ya que cada vez que se actualice la página en el navegador, cambiará.

- Debido a que la página se regenera en el servidor cada vez que se solicita, la expresión crea un objeto Date nuevo y la hora de creación se actualiza.

COMO ESCRIBIR EXPRESIONES JSP

- Si quisiéramos que la página fechatiempo.jsp se actualizase cada segundo, esto es, que actuara como un reloj:

```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC
 "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <title>Fecha y hora actual</title>
  <meta http-equiv="refresh"
content="1";url="http://localhost:8080/SegurosGenerales/fechatiempo.jsp">
 </head>
 <body>
 <!-- Fichero: fechatiempo.jsp -->
  <p>
  La fecha de hoy es
  <%= new java.util.Date() %>
  </p>
 </body>
</html>
```

DIRECTIVAS JSP

- En la JSP fechatiempo.jsp nos hemos referido a la clase java.util.Date utilizando su nombre completo (paquetes incluidos).

- Cuando utilizamos código Java, solemos usar sentencias *import* para poder referenciar a las clases sin tener que utilizar los nombres completos de los paquetes.

- Las páginas JSP también tienen una sentencia *import*, en forma de directiva JSP.

DIRECTIVAS JSP

- Las directivas son mensajes para el motor JSP.

- Aparecen siempre en la parte superior de la página JSP y son siempre incluidas entre los símbolos **<%@** y **%>**.

- Existen tres tipos de directivas: **page**, **include** y **taglib**.

  - Por ejemplo, haríamos lo siguiente para importar todas las clases de *java.util*:

```
<%@ page import="java.util.*" %>
```

DIRECTIVAS JSP

- Aplicamos esto que acabamos de ver a la JSP fechatiempo.jsp:

```
<%@ page import="java.util.*" %>
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC
 "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <title>Fecha y hora actual</title>
  <meta http-equiv="refresh"
content="1";url="http://localhost:8080/SegurosGenerales/fechatiempo.jsp">
 </head>
 <body>
 <!-- Fichero: fechatiempo.jsp -->
  <p>
  La fecha de hoy es
  <%= new Date() %>
  </p>
 </body>
</html>
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

- Los ejemplos vistos hasta el momento sólo generaban desde el servidor código HTML de salida hacia el cliente.

- Ahora vamos a recibir en el servidor un formulario desde un cliente (navegador). A esto se le conoce como petición HTTP.

- Este formulario será procesado en el servidor y se devolverá una respuesta al cliente. A esto se le conoce como respuesta HTTP.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

- Uno de los aspectos más útiles de las peticiones HTTP es que pueden pasarse datos como parámetros desde el cliente al servidor.

- Cuando la petición es de tipo GET los datos se incluyen en el URL en el navegador. Cuando la petición es de tipo POST los datos se incluyen en el cuerpo de la petición.

- Los propios datos, que provienen generalmente de formularios HTML, los rellena el usuario y los envía al servidor cuando se pulsa el botón **submit**.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

EJEMPLO

- Vamos a crear un formulario HTML para que un cliente de la compañía de seguros de nuestra empresa declare un siniestro.

- El formulario pedirá al usuario:

  - Número de póliza
  - Tipo de reclamación (continente o contenido)
  - Cantidad aproximada de la reclamación
  - Una descripción general de la reclamación

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <title>Formulario de notificacion de siniestro</title>
 </head>
 <body>
 <!-- Fichero: reclamacion.htm -->
 <h1>Formulario de notificacion de siniestro</h1>
  <form action="reclamacionpag.jsp" method="get">
   <table>
    <tr>
     <td><label for="numeropoliza">Numero de poliza:</label></td>
     <td><input type="text" id="numeropoliza" name="numeroPoliza" />></td>
    </tr>
    <tr>
     <td><label for="reclamacion">Importe reclamado:</label></td>
     <td><input type="text" id="reclamacion" name="reclamacion" />></td>
    </tr>
```

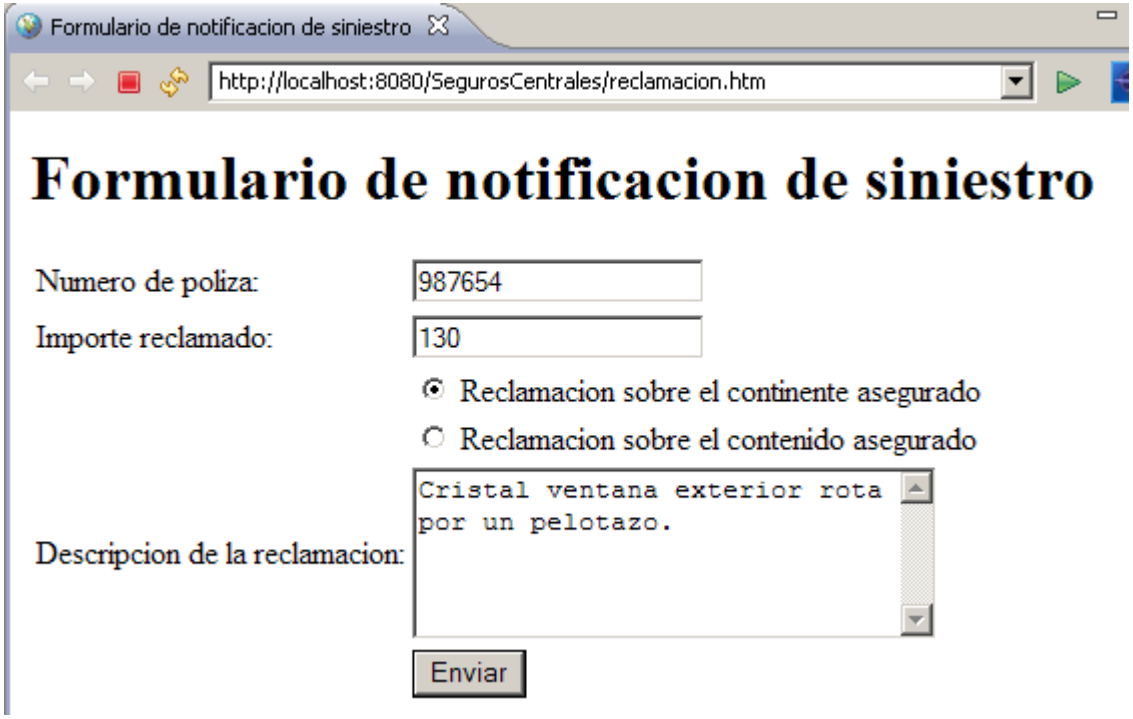
PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

(continuación del código)

```
<tr>
    <td></td><td>
    <input type="radio" name="tipo" id="continente" value="continente" />
    <label for="continente">Reclamacion sobre el continente asegurado</label>
    </td>
    </tr>
    <tr>
    <td></td><td>
      <input type="radio" name="tipo" id="contenido" value="contenido" />
      <label for="contenido">Reclamacion sobre el contenido asegurado</label>
</td>
</tr>
    <tr>
  <td><label for="descripcion">Descripcion de la reclamacion:</label></td>
  <td><textarea id="descripcion" name="descripcion" rows="5" cols="30"></textarea>
      </td>
    </tr>
    <tr>
     <td></td>
     <td><input type="submit" value="Enviar" />
     </td>
    </tr>
   </table>
  </form>
 </body>
</html>
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

- El formulario presenta el siguiente aspecto una vez rellenado por un cliente:



PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

- La parte fundamental de la página es el **formulario (<FORM …>**) y su atributo **action**, que define donde se enviarán los datos cuando se pulse el botón **submit**.

```
<form action="reclamacionpag.jsp" method="get">
```

- El atributo **method** establece el tipo de petición HTTP que se va a utilizar. Normalmente se utiliza POST, aunque en este ejemplo utilizamos GET, que puede ser más útil a la hora de entender cómo se incluyen los parámetros del formulario en una petición.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

<form action=*"reclamacionpag.jsp"* method=*"get"*>

- La acción destino será una página JSP.

- Vemos como la cadena URL relativa no está precedida por una barra oblicua (/), con lo que se supone que <u>el recurso está en la misma aplicación Web</u>.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

<form action=*"reclamacionpag.jsp"* method=*"get"*>

- Si le anteponemos una barra (/), también debemos indicar el nombre de la aplicación Web, por ejemplo:

<form action=*"/SegurosCentrales/reclamacionpag.jsp"* method=*"get"*>

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

- Los atributos **name** de los componentes en el formulario son muy importantes porque se usan para identificar los datos cuando se envían al servidor como parte de la petición HTTP.

- Por ejemplo, el valor elegido de los botones de selección es ‘tipo’ (el uso de mayúsculas o minúsculas importa en este caso).

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

Nota

- Para la mayoría de los elementos HTML, el atributo **id** ha reemplazado al atributo **name**, que se usaba en versiones anteriores de HTML.

- No obstante, los componentes de formulario son una excepción y pueden tener ambos atributos, **id** y **name**.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP
Parámetros y formularios

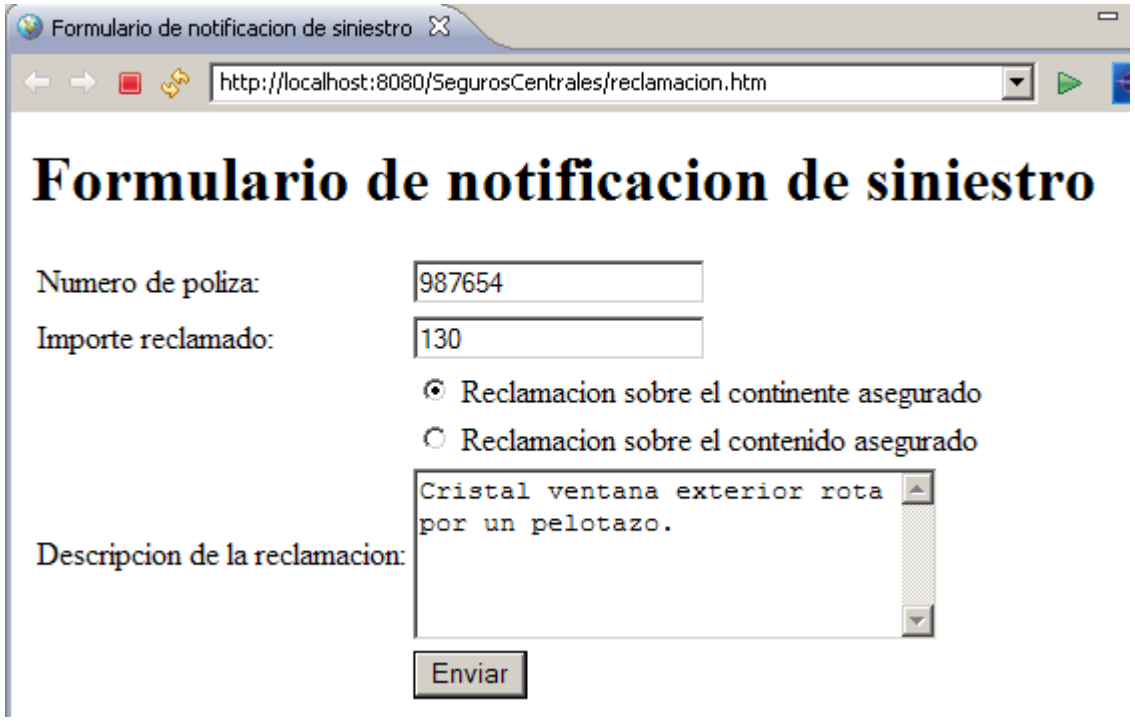
- Se pueden añadir tantos botones al formulario como se desee, pero siempre se debe tener un botón **submit** para que ejecute la acción y se envíen los datos del formulario al servidor.

```
<input type="submit" value="Enviar" />
```

- Hay que recordar que sólo el atributo **type** tiene que tener el valor **submit**. El atributo **value** es simplemente el texto que aparecerá en el botón.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de parámetros en peticiones HTTP



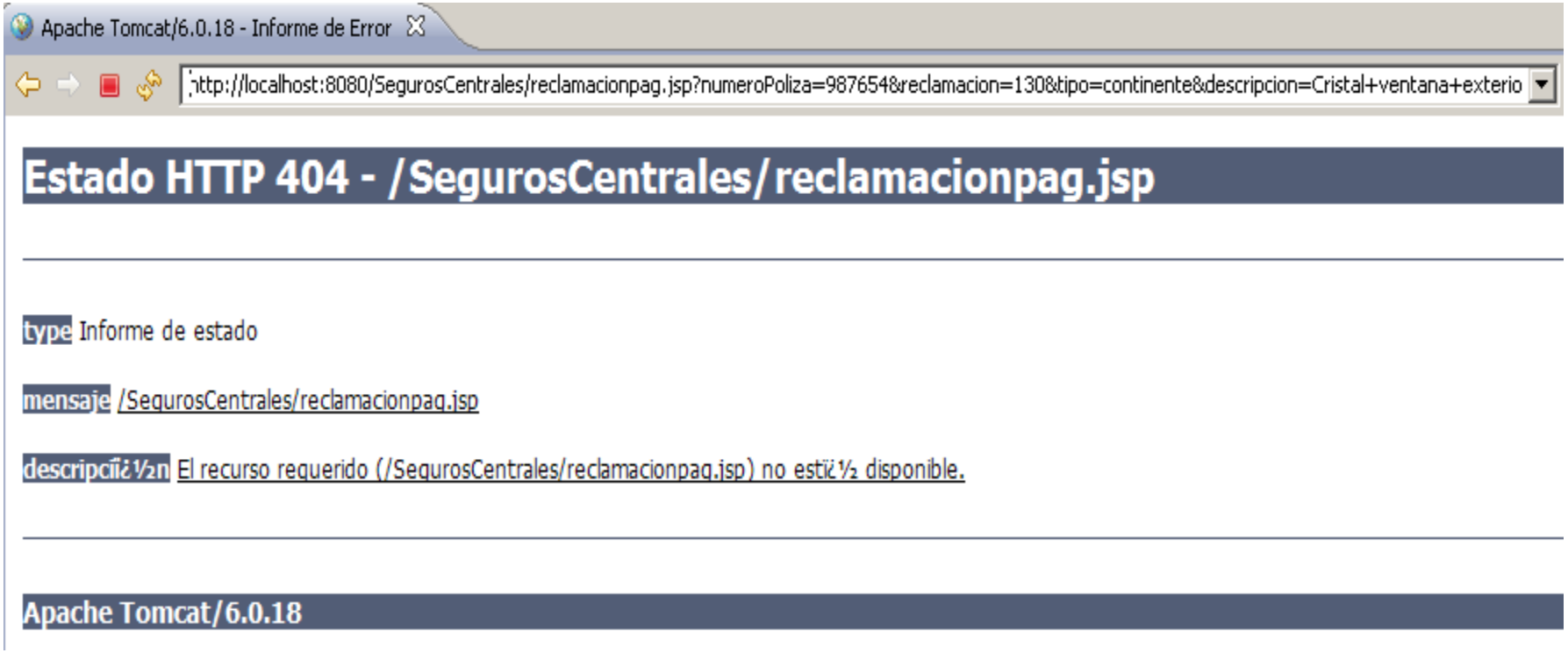
Cuando el usuario pulse el botón *Enviar* podremos ver en la barra de direcciones como se añaden los parámetros a la petición GET:

```
http://localhost:8080/SegurosCentrales/reclamacionpag.jsp?numeroPoliza=987
654&reclamacion=130&tipo=continente&descripcion=Cristal+ventana+exterior+r
ota+por+un+pelotazo.
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de parámetros en peticiones HTTP

- Obviamente el servidor mostrará un error, dado que aún no hemos creado la página JSP a la que se le está enviado la petición: ‘reclamacionpag.jsp’



PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de parámetros en peticiones HTTP

http://localhost:8080/SegurosCentrales/reclamacionpag.jsp**?numeroPoliza=987654&reclamacion=130&tipo=continente&descripcion=Cristal+ventana+exterior+rota+por+un+pelotazo**.

- Esto muestra los nombres de los parámetros que se pasan a la página JSP ‘reclamacionpag.jsp’, junto con sus valores.

- Necesitamos ahora crear tal página para que se encargue de procesar el contenido del formulario (vamos a implementar un arquitectura Modelo 1, esto es, sin servlet controlador).

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de parámetros en peticiones HTTP

- La página JSP ‘reclamacionpag.jsp’ utilizará un objeto predefinido llamado **request.**

- **request** es una instancia de la clase **HttpServletRequest** que tiene el método **getParameter()**.

- **getParameter()** tiene un parámetro de tipo **String** que representa el *nombre* (el valor del atributo *name*) de cada uno de los componentes del formulario HTML, y devuelve el valor de dicho componente del formulario.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de parámetros en peticiones HTTP

- Si el nombre del parámetro no coincide con ninguno de los elementos de la solicitud (petición), entonces el método devuelve **null**.

- Por ejemplo, para conseguir el valor del número de póliza en la JSP ‘reclamacionpag.jsp’ haríamos lo siguiente:

```
String numPoliza=request.getParameter("numeroPoliza");
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Los scriptlets y las páginas JSP

- ¿Qué código Java debemos utilizar para recuperar y procesar todos los valores de los parámetros?

- Primero veremos una solución mediante scriplets y más adelante una más actual mediante XML.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Los scriptlets y las páginas JSP

- A diferencia de lo que ocurre en las expresiones, donde puede haber únicamente una expresión Java, los scriptlets pueden tener varias líneas de código:

```
<%
      Código Java
      Código Java
      Código Java
%>
```

- Los scriptlets que escribamos en las páginas JSP serán código Java normal, como el que utilizamos cuando escribimos clases Java.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Los scriptlets y las páginas JSP

- Las páginas JSP disponen de unos objetos predefinidos, siendo los más importantes:
  - **out:** Es el objeto JSPWriter con el que escribir el contenido de la respuesta.
  - **request:** El objeto HttpServletRequest que nos permite acceder al objeto de la solicitud HTTP.
  - **response:** El objeto HttpServletResponse que nos da acceso al objeto de la respuesta HTTP.
  - **session:** Es el objeto HttpSession que representa la sesión de un usuario.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Los scriptlets y las páginas JSP

- A continuación se muestra el scriptlet que recupera los valores de los cuatro componentes del formulario HTML a partir de la petición y los almacena en variables de tipo String:

```
<%
String numeroPoliza=request.getParameter("numeroPoliza");
String reclamacion=request.getParameter("reclamacion");
String tipo=request.getParameter("tipo");
String descripcion=request.getParameter("descripcion");
%>
```

- Nótese como el tratamiento de los botones de opción del formulario (continente o contenido) es único, ya que comparten el mismo nombre (tipo)

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Procesamiento de los datos del formulario en la capa del controlador

- En una arquitectura ‘Modelo 1’ las páginas JSP actúan tanto de ‘controlador’ como de ‘vista’.

- Tareas de un rol ‘controlador’:

  - Si tenemos en cuenta que todos los parámetros del objeto **request** se devuelven como objetos **String**, a veces necesitaremos realizar conversiones a tipos de datos más adecuados.

  - En nuestro caso el importe de la póliza es un número real.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Procesamiento de los datos del formulario en la capa del controlador

- Tendremos que hacer algo así:

```
// Conversión del atributo reclamacion a objeto tipo double
try {
   double valorReclamacion=
   Double.parseDouble(reclamacion);
}
catch (NumberFormatException e)
{
   e.printStackTrace();
}
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Presentación de los datos del cliente en la capa de la vista

- Vamos a avanzar un poco más: haremos que la página JSP ‘reclamacionpag.jsp’ devuelva al cliente los mismos datos que ha enviado en el formulario al servidor (un acuse de recibo).

- Como detalle, mostraremos el importe de la reclamación formateado con dos decimales y los separador de millar con punto.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Presentación de los datos del cliente en la capa de la vista

El código de 'reclamacionpag.jsp' es el siguiente:

```
<!-- Fichero: reclamacionpag.jsp -->
<%@ page import="java.text.NumberFormat"%>
<%
String numeroPoliza=request.getParameter("numeroPoliza");
String reclamacion=request.getParameter("reclamacion");
String tipo=request.getParameter("tipo");
String descripcion=request.getParameter("descripcion");
//Convertir reclamacion a double

 double valorReclamacion = 0.0;
 try {
 valorReclamacion=
 Double.parseDouble(reclamacion);
 }
 catch(NumberFormatException e)
 {
  e.printStackTrace();
  System.out.println(reclamacion + " no es un número válido");
 }
 %>
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Presentación de los datos del cliente en la capa de la vista

(continuación)

```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC
 "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head><title>Detalles de la reclamacion del siniestro</title></head>
 <body>

  <h1><p>Gracias por realizar su reclamacion</p></h1>
  <p>Los detalles enviados</p>
  <p>Numero de reclamacion: <%= numeroPoliza %>

   <%
    NumberFormat formatoReal=NumberFormat.getCurrencyInstance();
    String valorReclamacionFormateado=
    formatoReal.format(valorReclamacion);
   %>

   <br /> Importe de la reclamacion: <%= valorReclamacionFormateado %>
   <br /> Tipo de reclamacion asegurada: <%= tipo %>
   <br /> Descripcion de la reclamacion: <%= descripcion %>
  </p>
 </body>
</html>
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Presentación de los datos del cliente en la capa de la vista

- Por tanto, cuando el cliente pulse sobre *Enviar* obtendrá:

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de datos desde un formulario HTML

- Las peticiones POST se utilizan para enviar los datos de los formularios que se almacenarán en el servidor o se utilizarán en otros procesos que intentarán cambiar el estado del servidor.

- Un ejemplo sería el envío de un formulario de pedido a un servidor Web de una aplicación de comercio electrónico.

- En un caso así, enviar la misma información dos veces puede desencadenar que se realicen dos pedidos en vez de uno.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de datos desde un formulario HTML

- En cambio, las peticiones GET pueden repetirse y no provocar ningún cambio en el estado del servidor, dado que se utilizan cuando no se debe modificar el estado del servidor.

- Por ejemplo, sería el caso de solicitar información sobre las modalidades de contratación de seguros de hogar existentes.

- Una petición así no modifica nada en el servidor, es decir, no implica ninguna transacción, tan sólo, la generación de un listado.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de datos desde un formulario HTML

- El método GET no es el adecuado para enviar los datos del formulario.

- Los motivos son variados:

  - GET incluye los parámetros en el URL, lo que puede producir problemas de seguridad.

  - GET tiene limitado su tamaño a 255 caracteres.

  - GET no admite el envío de contenido binario.

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de datos desde un formulario HTML

- Por todas estas razones, deberemos utilizar el método POST para enviar datos en un formulario al servidor.

- Esto significa que debemos cambiar nuestra página ‘reclamacionpag.htm’ para que se utilice POST en lugar de GET:

```
<form action="reclamacionpag.jsp" method="post">
```

PROCESAMIENTO DE DATOS DE PETICIONES HTTP

Parámetros y formularios->Envío de datos desde un formulario HTML

- Cuando volvamos a ejecutar la aplicación veremos que al pulsar *Enviar*, el URL no contendrá los datos de los parámetros:

JEE

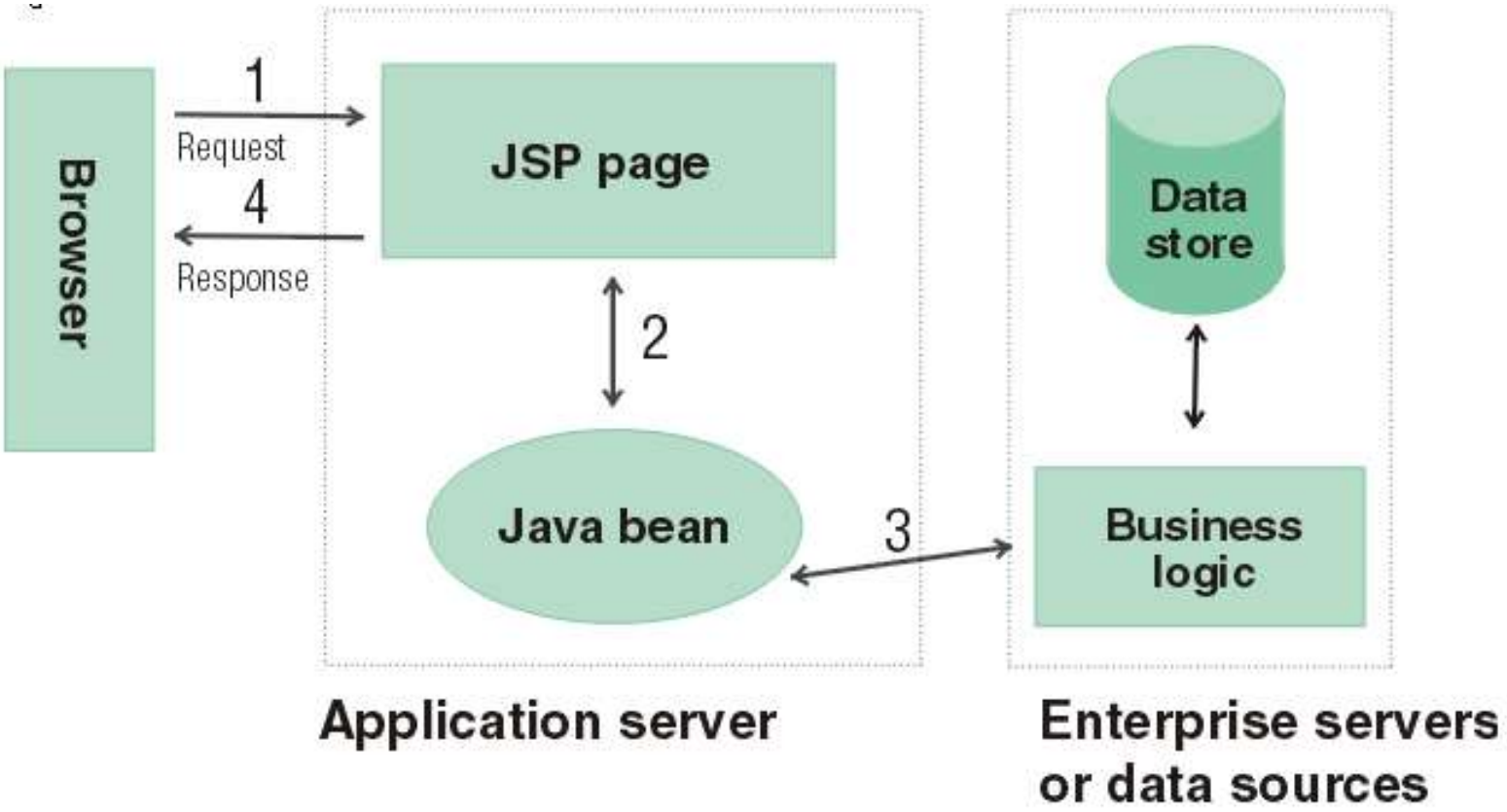
JSP

Uso de JavaBeans en JSP
Acciones JSP estándar
Arquitecturas Modelo 1 y Modelo 2 (MVC)
Control del flujo Web (introducción)

Parte 2

USO DE JAVABEANS EN PÁGINAS JSP

- Visión general de la arquitectura del Modelo 1:



USO DE JAVABEANS EN PÁGINAS JSP

- Un objeto JavaBean es un componente de programación Java que representa:

  - Un objeto DTO (Data Transfer Object) que se utiliza para encapsular datos relacionados (por ejemplo, los datos de un cliente) y transferirlos entre diferentes componentes de una aplicación Web.

  - O también una forma de procesar datos provenientes de la capa de la lógica de negocio (por ejemplo, la prima calculada que hay que pagar de un seguro).

USO DE JAVABEANS EN PÁGINAS JSP

- Siguiendo con nuestro ejemplo de aplicación vamos a crear un JavaBean llamado ReclamacionBean dentro del paquete beans.

- Es primordial que los JavaBeans se creen en paquetes diferentes del ‘default package’, ya que sino el motor JSP no los podrá procesar correctamente.

- Los JavaBeans deben implementar la interfaz java.io.Serializable para que puedan guardarse en disco, viajar por la red o enviarse por las distintas capas de la aplicación web.

USO DE JAVABEANS EN PÁGINAS JSP

- ReclamacionBean incluye propiedades que relacionan los datos que se envían desde la página reclamacion.htm: numeroPoliza, reclamacion, tipo y descripcion.

- El código para ReclamacionBean.java es el siguiente:

```
package beans;

import java.text.NumberFormat;
import java.io.Serializable;

public class ReclamacionBean implements Serializable {

    private static final long serialVersionUID = 1L;
    private String numeroPoliza;
    private double reclamacion;
    private String tipo;
    private String descripcion;
```

USO DE JAVABEANS EN PÁGINAS JSP

(continuación)

```
public ReclamacionBean() {
   super();
}

public String getNumeroPoliza() {
   return numeroPoliza;
}

public void setNumeroPoliza(String numeroPoliza) {
   this.numeroPoliza = numeroPoliza;
}

public String getReclamacion() {

   NumberFormat formatoReal = NumberFormat.getCurrencyInstance();

   String valorReclamacionFormateado =
           formatoReal.format(reclamacion);

   return valorReclamacionFormateado;
}
```

USO DE JAVABEANS EN PÁGINAS JSP

(continuación)

```
    public void setReclamacion(String importeReclamacion) {
       try {
              reclamacion = Double.parseDouble(importeReclamacion);
       } catch (NumberFormatException e) {
              e.printStackTrace();
              System.out
                .println(importeReclamacion + " no es un numero valido.");
       }
    }

    public String getTipo () {
       return tipo;
    }

    public void setTipo (String tipo) {
       this.tipo = tipo;
    }

    public String getDescripcion() {
       return descripcion;
    }
    public void setDescripcion(String descripcion) {
       this.descripcion = descripcion;
    }
}
```

USO DE JAVABEANS EN PÁGINAS JSP

- Podríamos usar en un scriptlet el JavaBean que acabamos de crear.

- Sin embargo, es mejor estrategia usar las etiquetas especiales que tienen las páginas JSP, también conocidas como acciones JSP estándar.

- Las acciones JSP estándar reducen la cantidad de código Java de nuestras páginas JSP, sustituyéndolo por etiquetas XML.

USO DE JAVABEANS EN PÁGINAS JSP
Etiquetas JSP para manejar JavaBean

- Los JavaBean pueden usarse en páginas JSP utilizando etiquetas especiales que hacen más fácil consultar y modificar sus propiedades.

- En el siguiente ejemplo reemplazaremos el código scriptlet existente con etiquetas JavaBean.

USO DE JAVABEANS EN PÁGINAS JSP
Etiquetas JSP para manejar JavaBean

- Estas etiquetas (acciones JSP) nos permiten crear una nueva instancia de un JavaBean o acceder a uno que ya haya sido creado dentro de la aplicación Web:

```
<jsp:useBean id="reclamacionBean" class="beans.ReclamacionBean" />
```

- Incluso las propiedades de los JavaBean pueden rellenarse automáticamente a partir de las diferentes peticiones HTTP:

```
<jsp:setProperty name="reclamacionBean" property="*" />
```

Nota: El atributo `name` en un elemento `jsp:setProperty` debe coincidir con el atributo `id` de algún elemento `jsp:useBean` anterior.

USO DE JAVABEANS EN PÁGINAS JSP
Escritura de propiedades del bean en el objeto de respuesta

- A continuación vemos un versión revisada de la JSP ‘reclamacionpag.jsp’.

- En este ocasión se utiliza JavaBean para:

  - Recuperar los datos de los parámetros del formulario HTML.

  - Validar y dar formato al importe de la reclamación (esto se hace en la clase del JavaBean).

  - Escribir la respuesta de vuelta al cliente.

USO DE JAVABEANS EN PÁGINAS JSP
Escritura de propiedades del bean en el objeto de respuesta

- A continuación vemos un versión revisada de la JSP ‘reclamacionpag.jsp’ (renombramos la otra para comparar después):

```
<jsp:useBean id="reclamacionBean" class="beans.ReclamacionBean" />
<jsp:setProperty name="reclamacionBean" property="*" />

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
<head><title>Detalles de la reclamacion del siniestro</title></head>
<body>

<p><h1>Gracias por realizar su reclamacion</h1></p>

<p>Los detalles enviados</p>
<p>Numero de reclamacion: <jsp:getProperty name="reclamacionBean"
property="numeroPoliza" /> <br />
Importe de la reclamacion: <jsp:getProperty name="reclamacionBean"
property="reclamacion" /> <br />
Tipo de reclamacion asegurada: <jsp:getProperty name="reclamacionBean"
property="tipo" /> <br />
Descripcion de la reclamacion: <jsp:getProperty name="reclamacionBean"
property="descripcion" />></p>
</body>
</html>
```

USO DE JAVABEANS EN PÁGINAS JSP
Escritura de propiedades del bean en el objeto de respuesta

- Al utilizar JavaBean:
  - Ya no es necesario incluir scriptlets o directivas de página que importen la clase NumberFormat.
  - Obtendremos un páginas JSP más sencilla, limpia y sin código Java.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Aunque nuestra aplicación Web funciona, no está empleando el mejor modelo.

- El motivo es que la página ‘reclamacionpag.jsp’ tiene demasiadas responsabilidades: está actuando de controlador y de vista.

- De ‘controlador’ porque tiene que procesar la petición y de ‘vista’ porque tiene generar la respuesta para el cliente.

LA ARQUITECTURA DEL MODELO 2 EN JSP

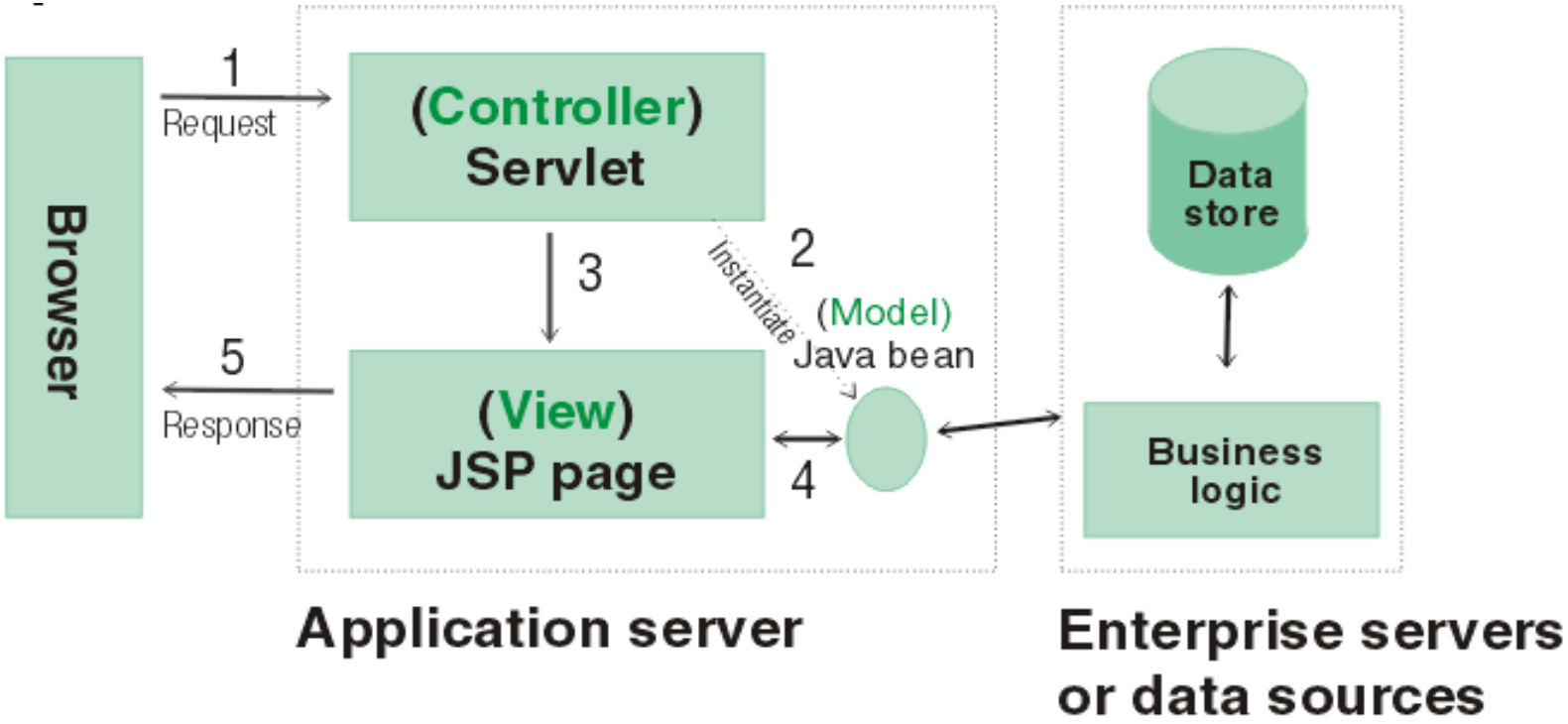
- **Llegados a este punto creamos un nuevo proyecto que llamaremos SegurosCentralesModelo2.**

- **Copiamos a este nuevo proyecto los recursos del proyecto anterior:**

  - **El JavaBean**
  - **El formulario de reclamación**
  - **La JSP de reclamacionpag.jsp**

LA ARQUITECTURA DEL MODELO 2 EN JSP

➢Para mejorar el diseño actual podemos aplicar el Modelo 2 (aunque lo haremos sólo con páginas JSP y no con un servlet controlador):



1. The browser sends a request to a servlet.
2. The servlet instantiates a Java bean that is connected to a database.
3. The servlet communicates with a JSP page.
4. The JSP page communicates with the Java bean.
5. The JSP page responds to the browser.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Utilizaremos una página para procesar la petición del cliente y otra para generar la vista de retorno.

- Con esta separación de responsabilidades, la página que procesa la petición delega en otras la generación de la vista, con lo que:

  - En caso de que los datos del cliente no sean correctos se le vuelve a reenviar el mismo formulario indicándole el motivo.

  - Se construye la siguiente página que debe ver el cliente cuando los datos sí sean correctos.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Queremos un patrón MVC, donde:
  - Los JavaBean representan el modelo.
  - La página del servidor que recibe los datos es el controlador.
  - Las páginas del servidor que construyen la respuesta son las vistas.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Para reenviar de una página JSP a otra (del controlador a las vistas) utilizaremos la etiqueta jsp:forward, que es una de las etiquetas de acciones estándar.

- Así, haremos lo siguiente para que la página que actúa de controlador llame a la página que genera la respuesta:

```
<jsp:forward page="reclamacionRespuestapag.jsp" \>
```

- Después de ejecutar esta acción, la página actual ya no podrá hacer nada, puesto que el control se traspasa al otro componente Web.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- En la arquitectura Modelo 2 las dos páginas utilizan el mismo JavaBean ‘ReclamacionBean’.

- La primera (el controlador) crea y rellena con datos el bean y la segunda (la vista) lo recupera y muestra su contenido.

- *Pero atención: la visibilidad predeterminada de un JavaBean es page, lo que significa que es local al JSP dentro del que se ha creado.*

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Con la visibilidad por defecto (page) de un JavaBean, la vista no podrá recuperarlo (pues se creo en el controlador).

- Para resolver el problema cambiaremos la visibilidad del JavaBean a request, lo que significa que estará disponible para el ciclo de vida de una petición en curso.

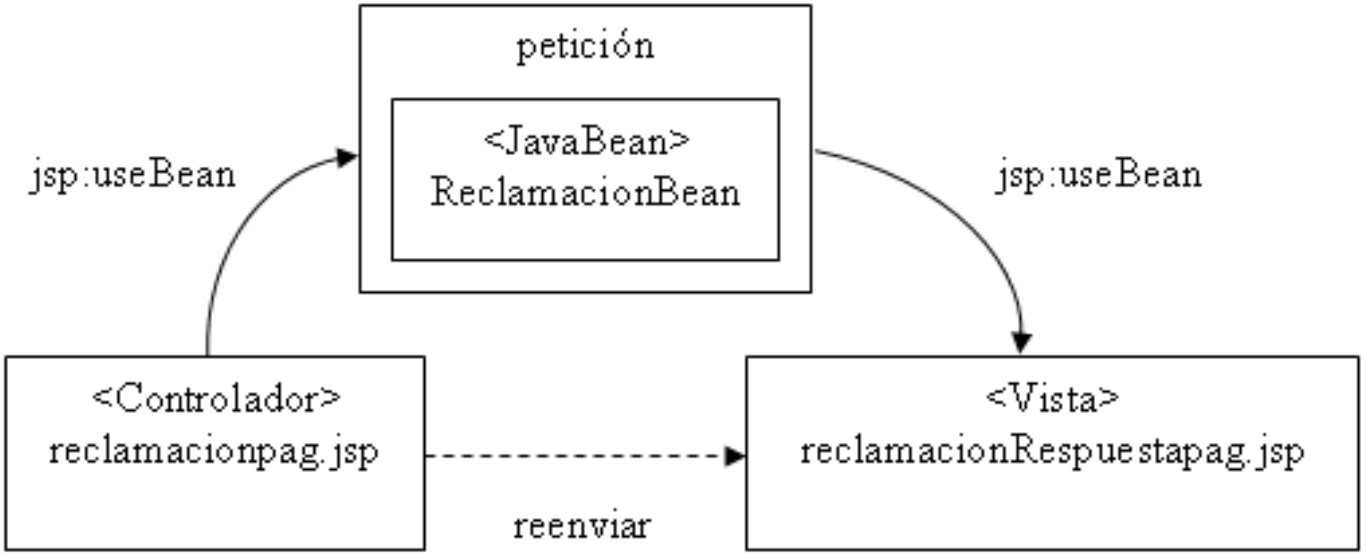
Nota: Una petición permanece disponible en el servidor hasta que el objeto respuesta se envíe de nuevo al cliente.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Por tanto, en el controlador 'reclamacionpag.jsp' haremos la modificación pertinente:

```
<jsp:useBean id="reclamacionBean" class="beans.ReclamacionBean"
scope="request" />
```

- Con lo anterior la página 'reclamacionRespuestapag.jsp' podrá acceder al bean 'ReclamacionBean' y generar la respuesta.



LA ARQUITECTURA DEL MODELO 2 EN JSP

- Este el nuevo código para ‘reclamacionpag.jsp’:

```
<!-- Fichero: reclamacionpag.jsp -->
<jsp:useBean id="reclamacionBean" class="beans.ReclamacionBean"
scope="request" />
<jsp:setProperty name="reclamacionBean" property="*" />

<jsp:forward page="reclamacionRespuestapag.jsp"></jsp:forward>
```

- Esta página crea el bean, inserta los datos del formulario del cliente en las propiedades del bean y lo reenvía a la segunda JSP (a la vista).

- La vista utilizará la etiqueta jsp:useBean pero el bean no será instanciado porque ya existe; sencillamente se usará.

LA ARQUITECTURA DEL MODELO 2 EN JSP

- Este el código para ‘reclamacionRespuestapag.jsp’:

```
<!-- Fichero: reclamacionRespuestapag.jsp -->
<jsp:useBean id="reclamacionBean" class="beans.ReclamacionBean"
scope="request" />
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
<head>
<title>Detalles de la reclamacion del siniestro</title>
</head>
<body>
<h1>Gracias por realizar su reclamacion</h1>
<p>Los detalles enviados</p>
<p>Numero de reclamacion: <jsp:getProperty name="reclamacionBean"
property="numeroPoliza" /> <br />
Importe de la reclamacion: <jsp:getProperty name="reclamacionBean"
property="reclamacion" /> <br />
Tipo de reclamacion asegurada: <jsp:getProperty name="reclamacionBean"
property="tipo" /> <br />
Descripcion de la reclamacion: <jsp:getProperty name="reclamacionBean"
property="descripcion" /></p>
</body>
</html>
```

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- **Llegados a este punto creamos un nuevo proyecto que llamaremos SegurosCentralesGestionFlujo.**

- **No necesitaremos nada del proyecto anterior.**

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- Hasta el momento, los ejemplos vistos sólo han cubierto un ciclo petición-respuesta.

- En las aplicaciones reales los usuarios navegan por diferentes ciclos petición-respuesta, que en su conjunto forman el flujo de la aplicación.

- Cada flujo dirige al usuario por las diferentes opciones según las decisiones elegidas.

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- Vamos a ver un ejemplo en que el cliente tiene la opción de obtener un presupuesto para asegurar:
  - Su vivienda (el continente).
  - Una póliza de daños de accidentes (lo llamaremos el contenido). Aquí podría incluirse otro contenido, como joyas, cuadros, etc., pero no lo vamos a hacer.
  - Ambas, continente y contenido.

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- Para realizar esta funcionalidad tendremos que mantener la sesión del usuario durante los distintos ciclos petición-respuesta.

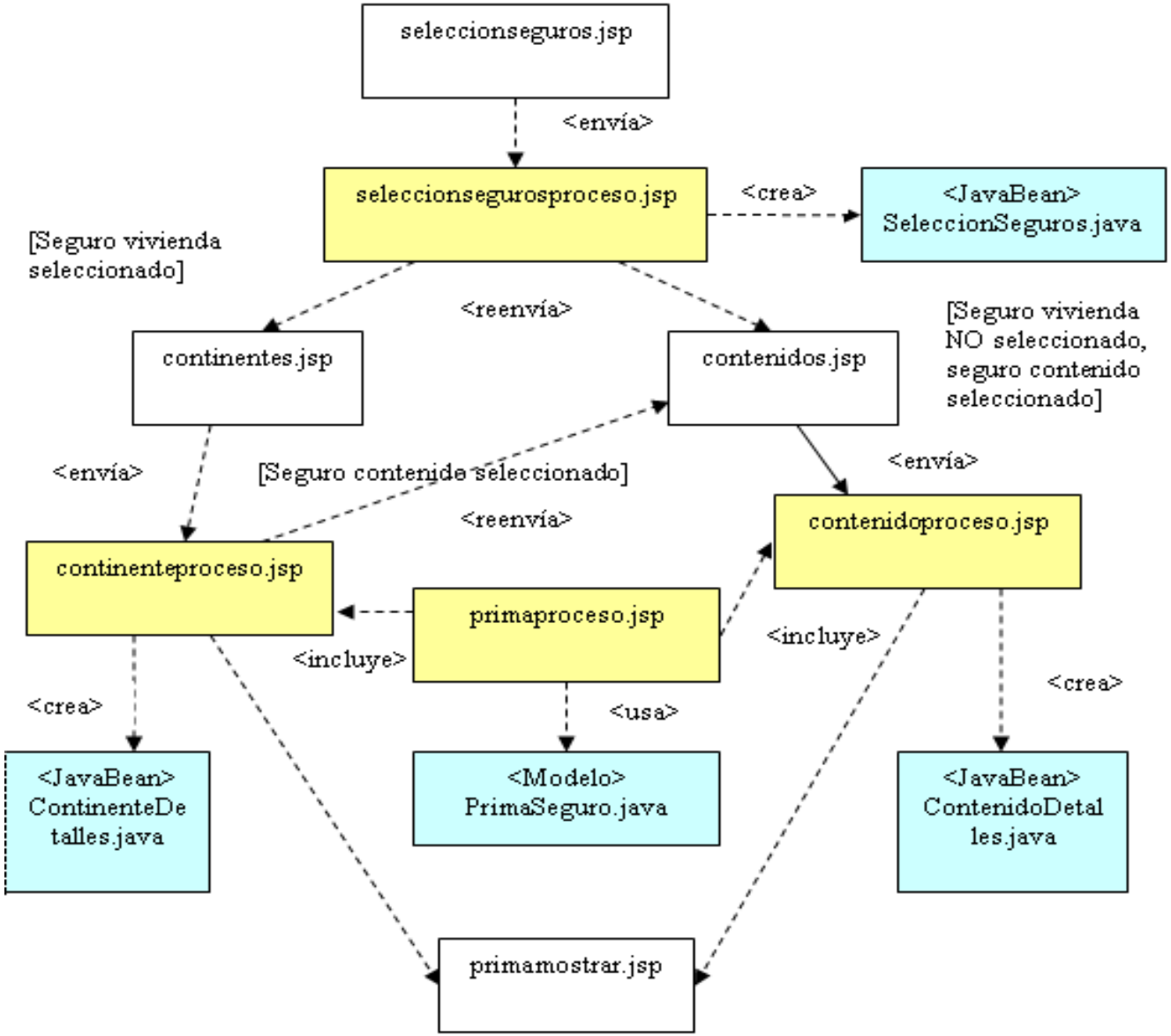
- Además tendremos que utilizar varios JavaBean para cada paso del flujo de la aplicación.

- No vamos a solicitar los datos personales del cliente (damos por hecho que ya los tenemos), tan sólo tiene que indicar qué tipos de seguros quiere.

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- El flujo de la aplicación:

```
blanco: vistas
amarillo: controlador
azul: modelo
```



GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- Fijémonos en el papel que desempeña ‘primaproceso.jsp’ dentro del flujo de la aplicación.

- ‘primaproceso.jsp’ se usa para contener procesos que son comunes a ‘continenteproceso.jsp’ y a ‘contenidoproceso.jsp’.

- En vez de duplicar esta funcionalidad en cada página , lo que hacemos es utilizar su implementación incluyendo la misma JSP.

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- Después se reenviará la aplicación a ‘primamostrar.jsp’ (esto lo hará cualquiera de las dos JSP anteriores).

- La etiqueta para utilizar una acción include es parecida a la etiqueta de la acción forward:

```
<jsp:include page="primaproceso.jsp" />
```

GESTIÓN DEL FLUJO DE UNA APLICACIÓN WEB

- El código fuente de esta aplicación lo vamos a ir viendo paso a paso.

- No obstante, antes veremos en el siguiente capítulo de JSP como funcionan la sesiones en un servidor web.

JEE

JSP

Gestión de la sesión Web
Caso práctico control del Flujo Web

Parte 3

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- La cuestión es cómo mantener la información del cliente en el servidor a medida que nos movemos por el flujo de la aplicación.

- El problema es que en el momento en que el servidor envía la respuesta al cliente los datos que se hubiesen recibido del formulario se pierden.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- Para mantener los datos del cliente en el servidor entre diferentes ciclos petición-respuesta tenemos el objeto HttpSession.

- Por cada cliente que esté utilizando la aplicación Web tendremos un objeto HttpSession.

- Este objeto representa la sesión del cliente y sirve para almacenar información relativa al él (el estado de su póliza de seguros, de su carrito de la compra, etc) mientras que utiliza otras páginas Web de la aplicación.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- La sesión del cliente va a estar disponible mientras que el cliente no cierre el navegador o el servidor decida invalidar la sesión por algún motivo (tiempo de inactividad agotado, transacción efectuada,..).

- Toda página JSP dispone de un objeto ***session*** de la clase HtppSession.

- Un objeto session es básicamente como un mapa Hash que contiene una colección de pares clave-valor, donde las claves son cadenas de texto y los valores son objetos.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- Podemos añadir objetos dentro de una sesión utilizando el método ***setAttribute***, por ejemplo:

```
session.setAttribute("Usuario", objUsuario);
```

- Donde suponemos que tenemos un objeto llamado objUsuario.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- Podemos conseguir una referencia a un objeto de una sesión usando el método ***getAttribute***.

- El tipo devuelto de getAttribute es de tipo Object, por lo que tendremos que adaptar el valor devuelto al tipo apropiado.

- Si suponemos que nuestro primer ejemplo, objUsuario, es de tipo String, podríamos hacer lo siguiente:

```
String nombreUsuario=(String)session.getAttribute("Usuario");
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- Normalmente, ponemos datos en una sesión dentro de una página JSP (o de un servlet) y los recuperamos desde otra.

- El método getAttribute devuelve una referencia a un objeto y lo mantiene en la sesión.

- Para eliminar un objeto de la sesión, por ejemplo, cuando el cliente escoge eliminar algo del carrito de la compra, utilizamos el método ***removeAttribute***, pasándole el valor correspondiente:

```
Session.removeAttribute("clave");
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- Un aspecto importante de la gestión de las sesiones es que se pueden liberar recursos que no se necesiten más.

- Por ejemplo, cuando el cliente confirma la compra del carrito, ya no necesitamos más el carrito, por lo que debemos eliminarlo de la sesión, liberando así los recursos asociados del servidor.

- Para deshacerse de un objeto que se encuentre en la sesión, podemos utilizar el método ***invalidate***, con el que eliminaremos el objeto de la sesión junto con todos los objetos que en su interior se encuentre:

```
session.invalidate();
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- Cuando un cliente comienza una determinada interacción en el flujo de la aplicación (por ejemplo, empezar a comprar artículos), se ha de revisar si la sesión que estamos utilizando es nueva o ya existía.

- Para ello podemos utilizar el método ***isNew*** y comprobar si existe.

- En caso afirmativo, deberemos invalidar la sesión ya existente y empezar una nueva, dado que podríamos tener muchos problemas al mezclar información vieja con la actual.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- También puede ocurrir que en un determinado punto del flujo de la aplicación donde sólo se puede llegar si existe una sesión, la sesión se haya perdido.

- En ese caso tendremos que redirigir o reenviar al cliente al principio del flujo de la aplicación.

- Una redirección envía una respuesta al navegador que hace que se redirija a otra página, mientras que un reenvío directamente trata con alguno de los recursos del lado servidor.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

- De cualquier modo, lo que está claro en un caso así, es que el objeto de petición que se esté utilizando se desechará.

```
<% if (session.isNew()
{
%>
      <jsp:redirect page="seleccionseguros.jsp" />
<%
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB->JavaBean y sesiones

- Para que un JavaBean esté disponible en la sesión tendremos que ponerlo en este ámbito mediante el atributo scope:

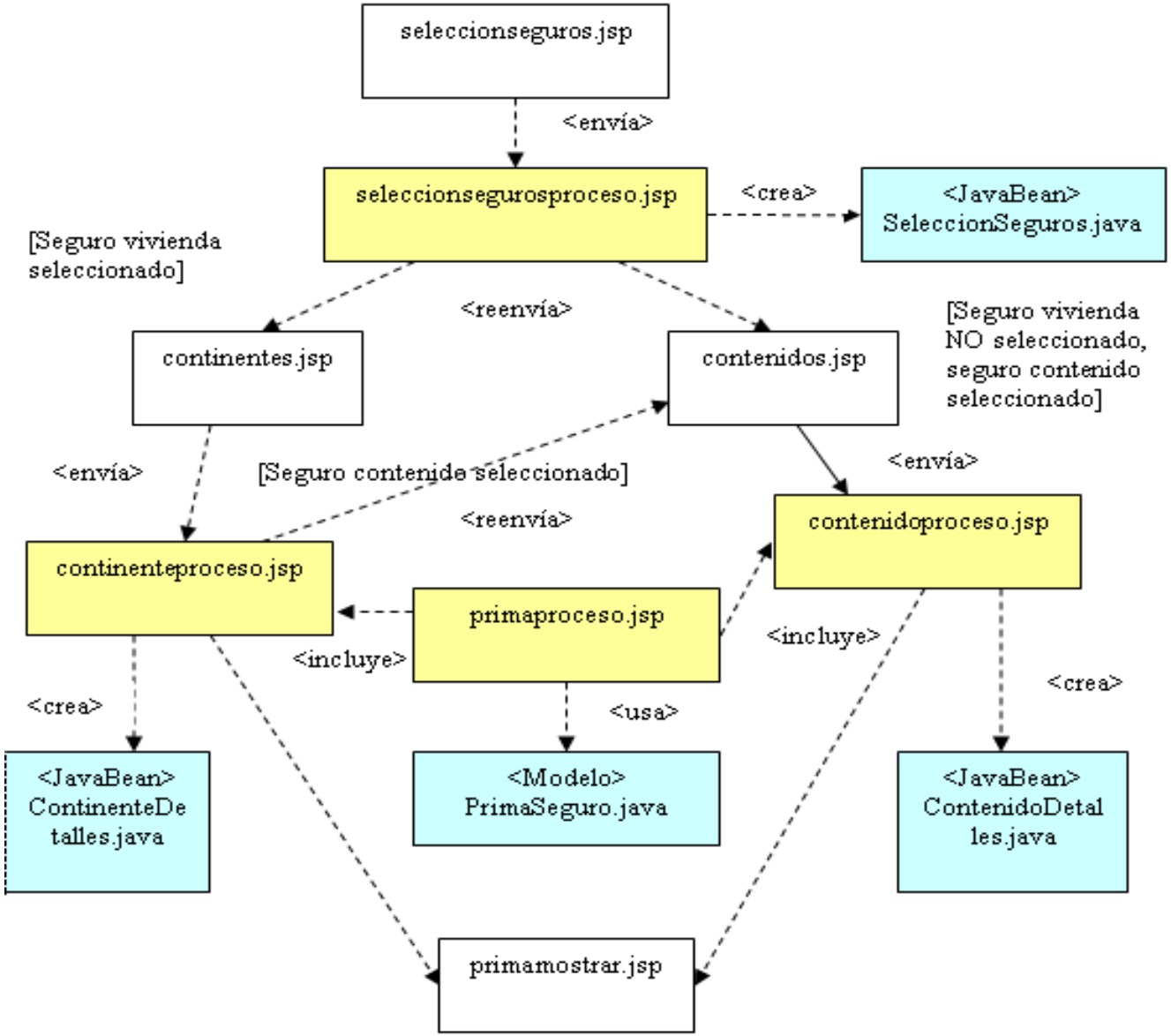
```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros"
scope="session" />
```

- De esta manera, el servidor buscará el bean en la sesión y si no existe entonces lo creará y lo dejará en la sesión.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Vamos a ahora a abordar esta aplicación



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Lo primero que necesitamos es una página de formulario JSP que pida al usuario los tipos de seguros para los que quiere un presupuesto.

- Para ello creamos la página ‘seleccionseguros.jsp’:

```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <link href="seguroscentrales.css" rel="stylesheet" type="text/css" />
  <title>SegurosCentrales - Solicitud de seguros</title>
 </head>
 <body>
 <!-- Fichero: seleccionseguros.jsp -->
  <h1>Seleccione su poliza</h1>
  <h2>Por favor, seleccione su seguro de edificios, de pertenencias o ambos,
marcando las casillas de verificacion</h2>
  <form action="seleccionsegurosproceso.jsp" method="post">
   <table>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

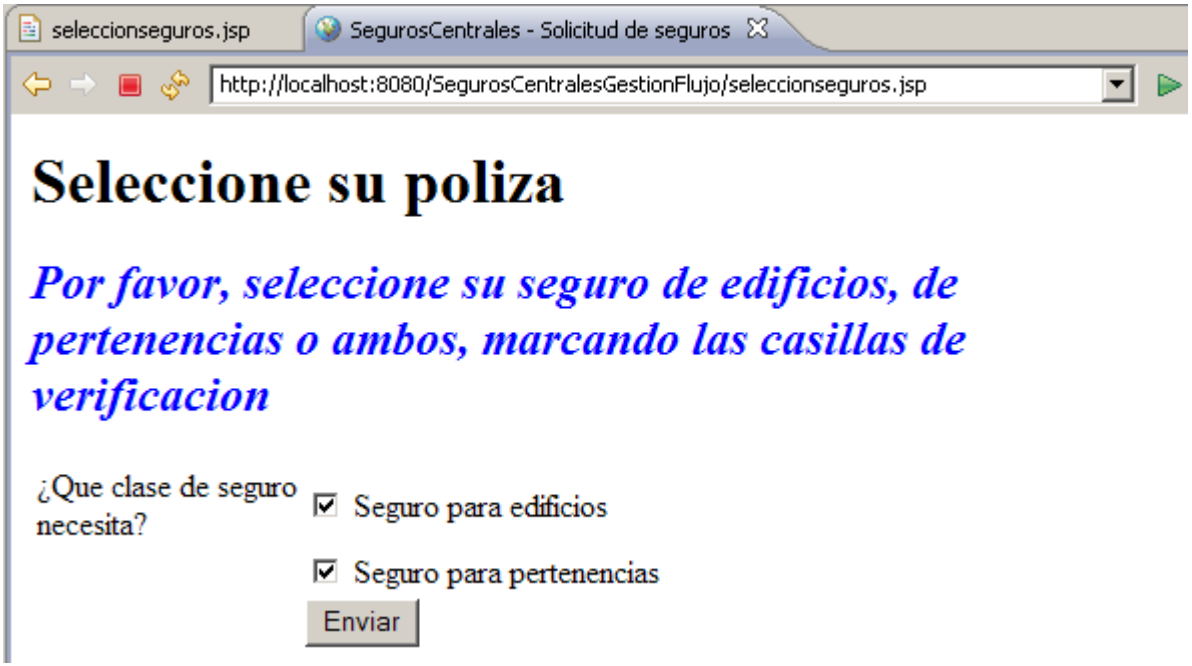
Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
  <tr><td>
    ¿Que clase de seguro <br /> necesita?
   </td>
   <td>
    <input type="checkbox" name="continente" checked="checked" />
    Seguro para edificios
   </td></tr>
  <tr>
   <td></td>
   <td>
    <input type="checkbox" name="contenido" checked="checked" />
    Seguro para pertenencias
   </td>
   </tr>
   <tr>
    <td></td>
    <td>
     <input type="submit" value="Enviar" />
    </td>
   </tr>
  </table>
 </form>
</body>
</html>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

➢En este formulario inicialmente los dos cuadros de selección se encuentran en estado checked, de forma que si el usuario no indica lo contrario se calcularán los dos tipos de presupuestos: continente (edificios) y contenido (pertenencias, accidentes, etc):



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Hay que prestar atención en los nombres de los cuadros de selección:

```
    <input type="checkbox" name="continente" checked="checked" />
     Seguro para edificios
...
     <input type="checkbox" name="contenido" checked="checked" />
     Seguro para pertenencias
```

- Ya que ambos tienen que coincidir con los nombres de las propiedades del JavaBean SeleccionSeguros.java, que almacenará la selección del usuario en el servidor, dentro del objeto sesión.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- A continuación se muestra el código del JavaBean SeleccionSeguros.java, el cual sólo tiene dos propiedades lógicas llamadas *continente* y *contenido*:

```
package beans;

import java.io.Serializable;

public class SeleccionSeguros implements Serializable {

private static final long serialVersionUID = 1L;
private boolean contenido;
private boolean continente;

   public SeleccionSeguros()
   {
       super();
   }
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
        // Este método se llama getContenido en vez de
        // isContenido para ser compatible con JSP
        public boolean getContenido() {
                return contenido;
        }

        public void setContenido(boolean contenido) {
                this.contenido = contenido;
        }

        // Este método se llama getContinente en vez de
        // isContinente para ser compatible con JSP
        public boolean getContinente() {
                return continente;
        }

        public void setContinente(boolean continente) {
                this.continente = continente;
        }

}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Cuando se envía el formulario al servidor, la representación de los cuadros de selección coincide con la clase String y su contenido consiste, o bien en el nombre del cuadro de selección, o bien en el valor *null* si es que no se ha seleccionado.

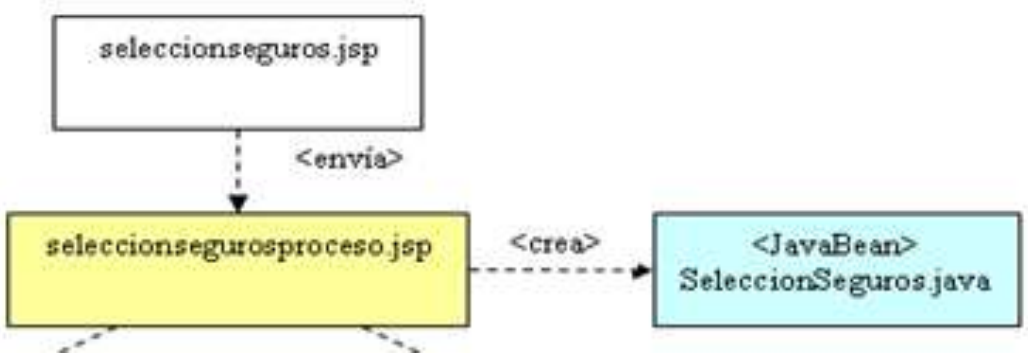
- En cambio, cuando usamos la etiqueta jsp:setProperty se realiza la conversión al tipo booleano de la variable del JavaBean de forma automática, por lo que no hay que incluir código alguno para convertir los valores provenientes del navegador.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- La página del servidor a la que se envía el formulario se llama ‘seleccionsegurosproceso.jsp’.

- De momento tenemos creado lo siguiente:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- ‘seleccionsegurosproceso.jsp’ tiene dos responsabilidades:
  - Crear y rellenar con datos una instancia del JavaBean SeleccionSeguros.
  - Enviar al usuario al siguiente formulario, en función de los seguros de los que quiera ver el presupuesto.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Código de ‘seleccionsegurosproceso.jsp’:

```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:setProperty name="selseguros" property="continente" value="false" />
<jsp:setProperty name="selseguros" property="contenido" value="false" />
<jsp:setProperty name="selseguros" property="*" />
<%
// Seleccionar la página destino en función del checkbox marcado

if(selseguros.getContinente())
{
%>
<jsp:forward page="continentes.jsp"/>
<%
}
else
{
 if(selseguros.getContenido())
 {
%>
<jsp:forward page="contenidos.jsp"/>
<%
 }
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Se ha de observar que la visibilidad del bean ‘selseguros’ es de tipo *sesión*, asegurando así que el bean permanezca en la sesión a lo largo de los distintos ciclos petición-respuesta.

```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros"
scope="session" />
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Los scriptlets se utilizan para elegir si se envía al usuario a la página ‘continentes.jsp’ o a ‘contenidos.jsp’ en el siguiente paso.

```
<%
// Seleccionar la página destino en función del checkbox marcado

if(selseguros.getContinente())
…
else
{
 if(selseguros.getContenido())
 {
…
 }
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Por último, podemos ver que se configuran las propiedades continente y contenido a false:

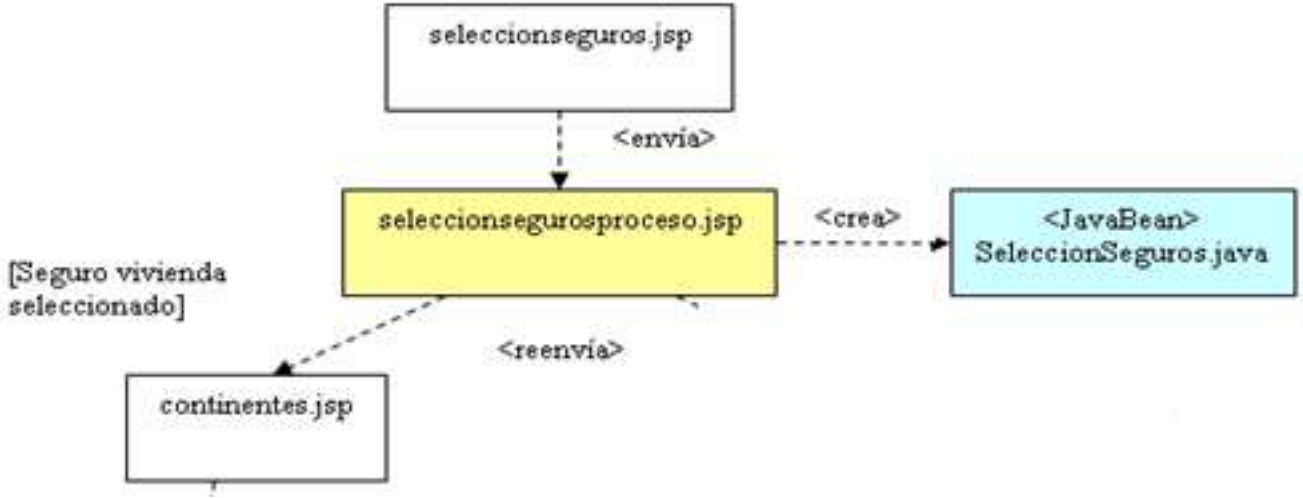
```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:setProperty name="selseguros" property="continente" value="false" />
<jsp:setProperty name="selseguros" property="contenido" value="false" />
<jsp:setProperty name="selseguros" property="*" />
```

- El motivo de tal inicialización es para solucionar el caso en que en la segunda o posteriores solicitudes de presupuestos por parte de un mismo usuario, si éste variase el tipo de seguros que desea ver, al estar el bean creado en la sesión mantendría los valores de la primera ejecución, dado que un cuadro de selección no marcado llega al servidor como null, valor que no variaría el estado de la propiedad del bean.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Si el usuario ha elegido la opción de obtener un presupuesto para un edificio (continente), la aplicación le redirigirá al formulario ‘continentes.jsp’, donde se le solicitará toda la información relativa a su vivienda:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- El código de ‘continentes.jsp’ :

```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <link href="seguroscentrales.css" rel="stylesheet" type="text/css" />
  <title>SegurosCentrales - Detalles del edificio</title>
 </head>
 <body>
 <!-- Fichero: continentes.jsp -->
  <h1>Detalles de la poliza de edificios</h1>
  <h2>Por favor, suministre la informacion sobre su vivienda en el siguiente formulario. A
continuacion pulse sobre <i>Enviar</i></h2>
  <form action="continentesproceso.jsp" method="post">
   <table>
    <tr>
     <td><label for="tipoPropiedad">Tipo de vivienda:</label></td>
     <td>
      <select name="tipoPropiedad" size=1>
       <option value="piso">Piso / Apartamento</option>
       <option value="adosada">Casa adosada</option>
       <option value="semiadosada">Casa semiadosada</option>
       <option value="aislada">Cuatro vientos</option>
      </select>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
</td>
  </tr>
  <tr>
   <td><label for="numeroHabitaciones">Numero de habitaciones:</label></td>
   <td>
    <select name="numeroHabitaciones" size=1>
     <option value="1">1</option>
     <option value="2">2</option>
     <option value="3">3</option>
     <option value="4">4</option>
     <option value="5">5</option>
     <option value="6">Mas de 5</option>
    </select>
   </td>
  </tr>
  <tr>
   <td><label for="fechaConstruccion">Fecha de construccion:</label></td>
   <td>
    <select name="fechaConstruccion" size=1>
     <option value="01/01/1900">Anterior a 1900</option>
     <option value="01/01/1901">1901 - 1950</option>
     <option value="01/01/1951">1951 - 1970</option>
     <option value="01/01/1971">1971 - 1990</option>
     <option value="01/01/1991">Posterior a 1990</option>
    </select>
   </td>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

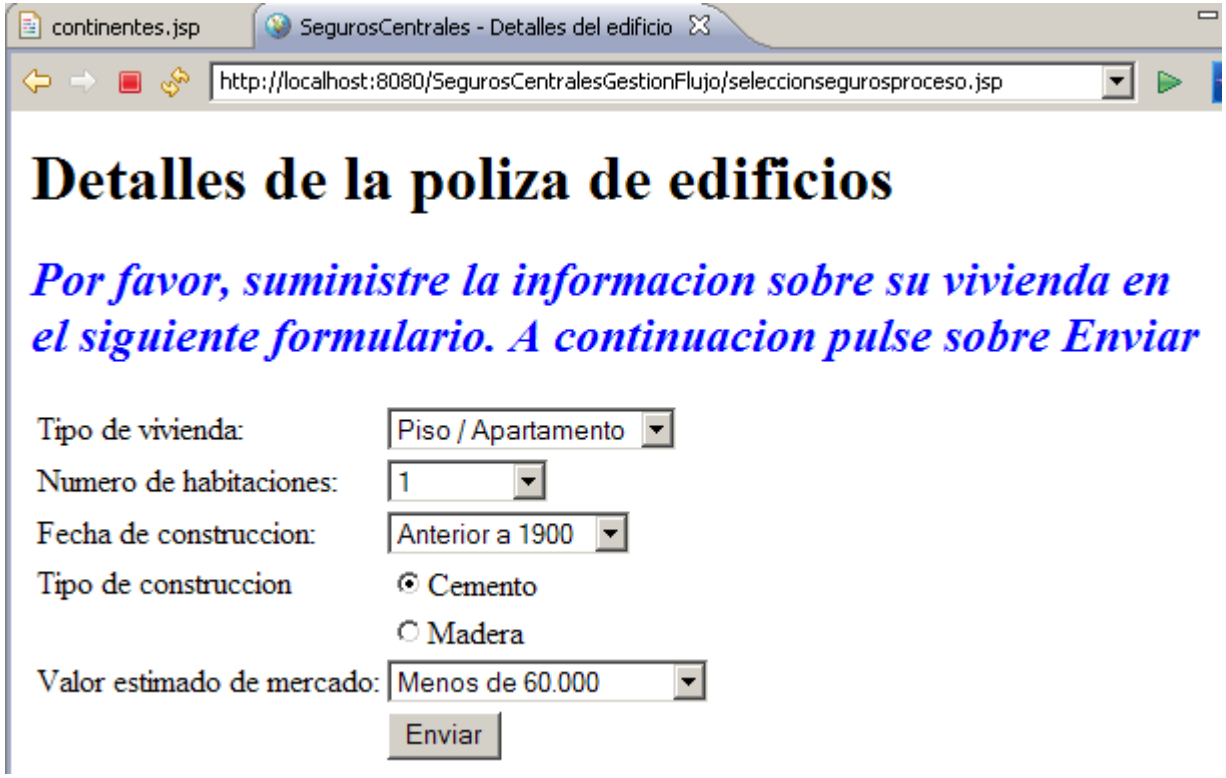
```
</tr>
   <tr>
    <td>Tipo de construccion</td>
    <td>
     <input type="radio" name="tipoConstruccion" value="cemento" checked="checked"
/>Cemento
    </td>
   </tr>
   <tr>
    <td>
     </td>
     <td>
      <input type="radio" name="tipoConstruccion" value="madera" />Madera
     </td>
    </tr>
    <tr>
     <td>Valor estimado de mercado:</td>
     <td>
      <select name="valorMercado" size=1>
       <option value="60000">Menos de 60.000</option>
       <option value="99000">60.000 - 99.000</option>
       <option value="149000">100.000 - 149.000</option>
       <option value="199000">150.000 - 199.000</option>
       <option value="500000">200.000 - 500.000</option>
       <option value="1000000">Por encima de 500.000</option>
      </select>
     </td>
    </tr>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
<tr>
     <td> </td>
     <td>
      <input type="submit" value="Enviar" />
     </td>
    </tr>
   </table>
  </form>
 </body>
</html>
```

El aspecto de este formulario es el siguiente:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- La información que se obtiene del usuario en este formulario consiste en:

  - El tipo de propiedad:

```
<select name="tipoPropiedad" size=1>
       <option value="piso">Piso / Apartamento</option>
       <option value="adosada">Casa adosada </option>
...
```

  - El número de habitaciones:

```
<select name="numeroHabitaciones" size=1>
       <option value="1">1</option>
       <option value="2">2</option>
...
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- La fecha de construcción:

```
<select name="fechaConstruccion" size=1>
       <option value="01/01/1900">Anterior a 1900</option>
       <option value="01/01/1901">1901 - 1950</option>
...
```

- El tipo de construcción:

```
<input type="radio" name="tipoConstruccion" value="cemento"
  checked="checked" />Cemento
    ...
<input type="radio" name="tipoConstruccion" value="madera" />Madera
```

- El valor de mercado:

```
<select name="valorMercado" size=1>
       <option value="60000">Menos de 60.000</option>
       <option value="99000">60.000 - 99.000</option>
...
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Todos estos componentes de formulario tienen su equivalente en el JavaBean ContinenteDetalles.java.

- El bean también incluye:

  - Un campo donde almacenar la cuota del seguro una vez que se ha calculado.

  - Los métodos set/get.

  - Un método que devuelve la cuota en formato de moneda.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- El código de ContinenteDetalles.java:

```
package beans;

import java.text.NumberFormat;
import java.io.Serializable;

public class ContinenteDetalles implements Serializable {

  private static final long serialVersionUID = 1L;

  private String tipoPropiedad;
  private int numeroHabitaciones;
  private String fechaConstruccion;
  private String tipoConstruccion;
  private int valorMercado;
  private double primaSeguro;

  public ContinenteDetalles() {
    super();
  }
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
public String getTipoPropiedad() {
  return tipoPropiedad;
}

public void setTipoPropiedad(String tipoPropiedad) {
  this.tipoPropiedad = tipoPropiedad;
}

public int getNumeroHabitaciones() {
  return numeroHabitaciones;
}

public void setNumeroHabitaciones(int numeroHabitaciones) {
  this.numeroHabitaciones = numeroHabitaciones;
}

public String getFechaConstruccion() {
  return fechaConstruccion;
}

public void setFechaConstruccion(String fechaConstruccion) {
  this.fechaConstruccion = fechaConstruccion;
}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
  public String getTipoConstruccion() {
    return tipoConstruccion;
  }

  public void setTipoConstruccion(String tipoConstruccion) {
    this.tipoConstruccion = tipoConstruccion;
  }

  public int getValorMercado() {
    return valorMercado;
  }

  public void setValorMercado(int valorMercado) {
    this.valorMercado = valorMercado;
  }

  public double getPrimaSeguro() {
    return primaSeguro;
  }

  public String getPrimaSeguroFormateada() {
    NumberFormat monedaFormato = NumberFormat.getCurrencyInstance();
    return monedaFormato.format(getPrimaSeguro());
  }

  public void setPrimaSeguro(double primaSeguro) {
    this.primaSeguro = primaSeguro;
  }
}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Vamos ahora con la página ‘continentesproceso.jsp’.

- Cuando recibe la petición HTTP desde la página del formulario, crea una instancia del bean ContinenteDetalles y modifica los valores de sus propiedades a partir de los datos de la petición.

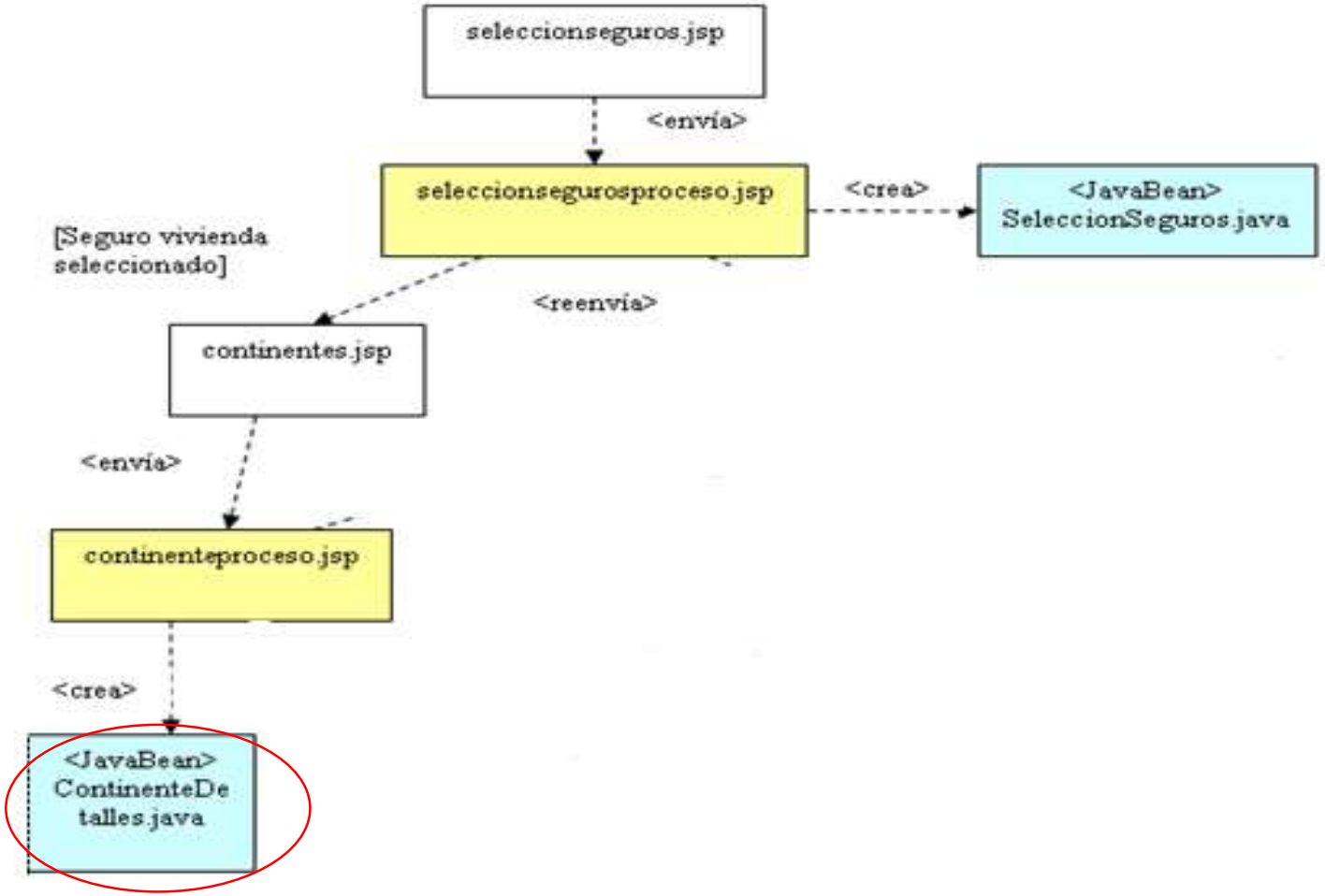
```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:useBean id="continente" class="beans.ContinenteDetalles" scope="session" />
<jsp:setProperty name="continente" property="*" />

<%
// seleccionar la siguiente página si también se requiere el seguro de contenidos
if(selseguros.getContenido()) {
%>
<jsp:forward page="contenidos.jsp" />
<%
}
else {
%>
<jsp:include page="primaproceso.jsp" />
<jsp:forward page="primamostrar.jsp" />
<%
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Gráficamente:



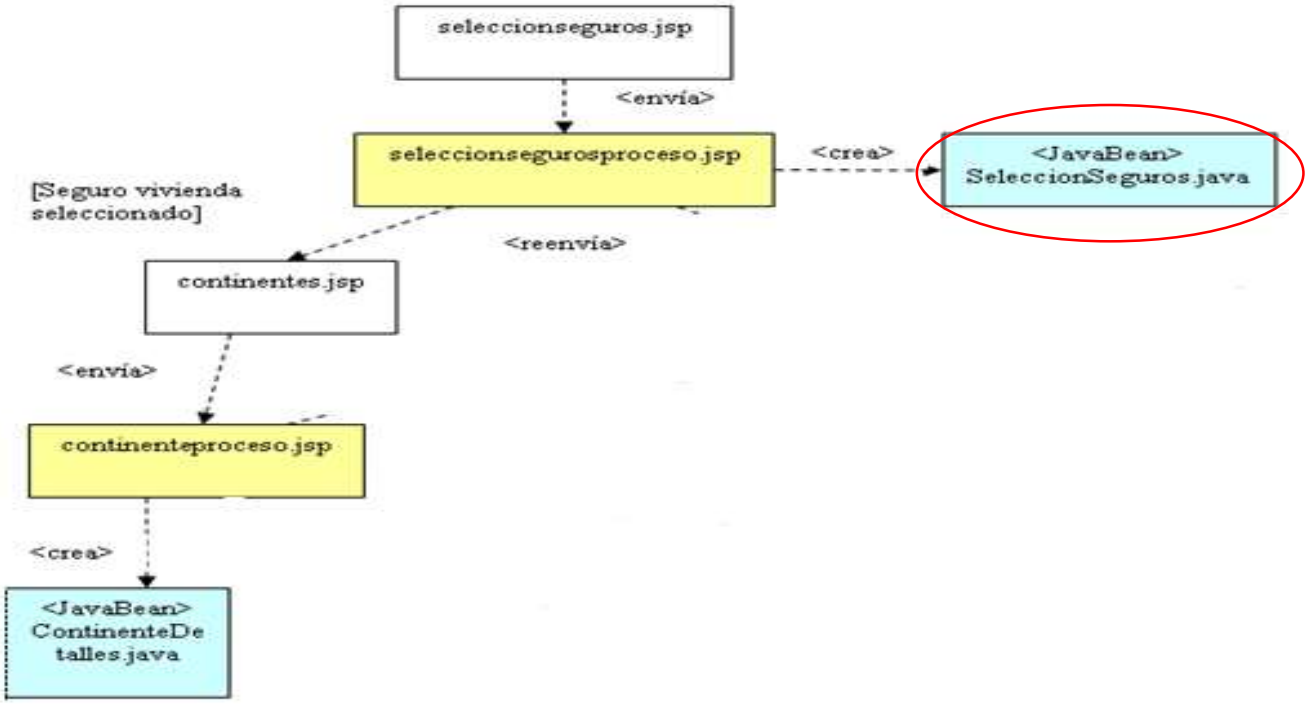
GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- También es necesario recuperar de la sesión el bean SeleccionSeguros para averiguar a dónde hay que ir en el siguiente paso (Esto se hace en la primera línea de código).

```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
...
```

Gráficamente:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Si el usuario ha elegido un presupuesto de contenido, la siguiente página tendrá que ser ‘contenidos.jsp’

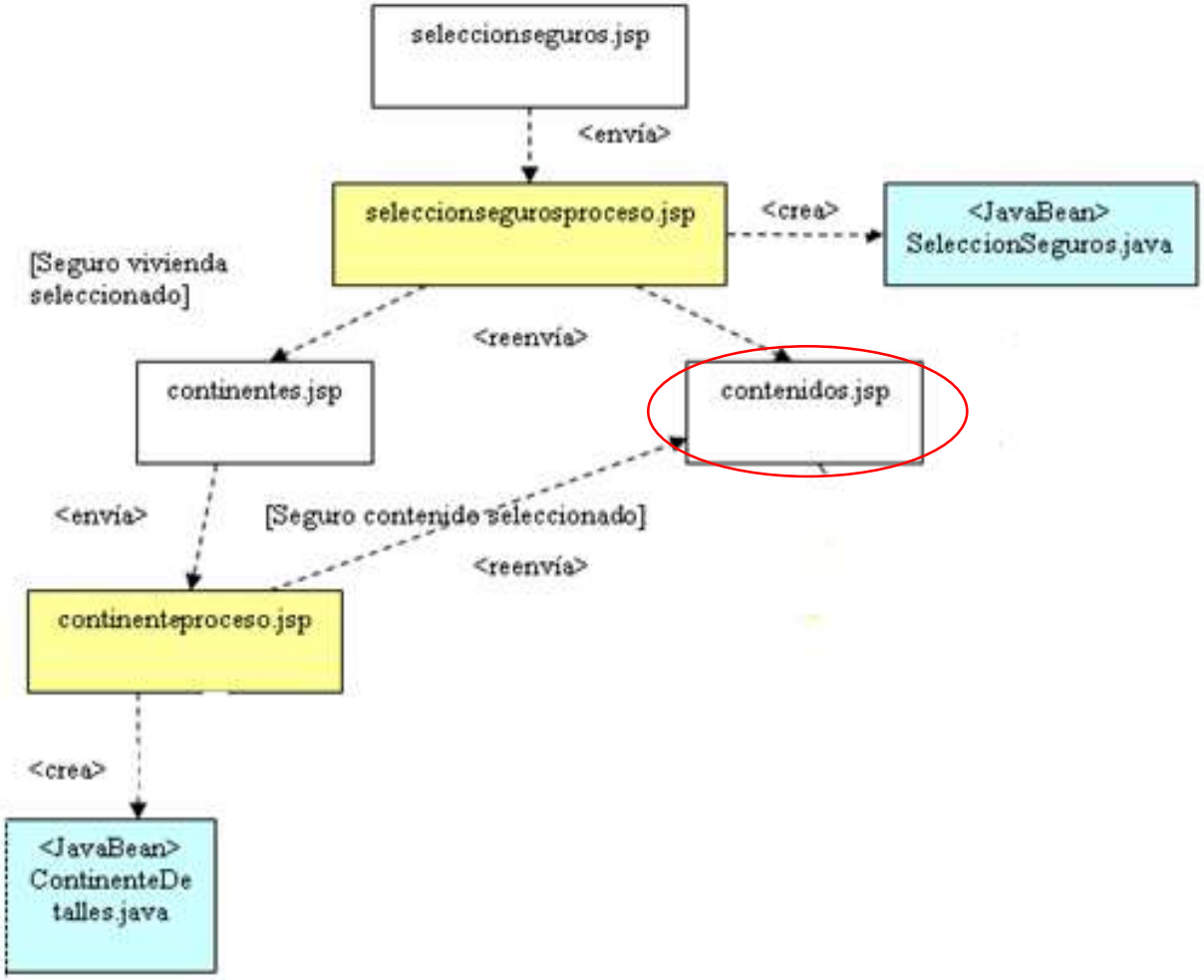
```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:useBean id="continente" class="beans.ContinenteDetalles" scope="session" />
<jsp:setProperty name="continente" property="*" />

<%
// seleccionar la siguiente página si también se requiere el seguro de contenidos
if(selseguros.getContenido()) {
%>
<jsp:forward page="contenidos.jsp" />
<%
}
else {
%>
<jsp:include page="primaproceso.jsp" />
<jsp:forward page="primamostrar.jsp" />
<%
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Gráficamente:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Pero si no se ha elegido el seguro de contenidos hay que calcular la cuota para el seguro de continente.

- Para ello, primero se incluye a la página ‘primaproceso.jsp’ y después se redirecciona hacia la página ‘primamostrar.jsp’ para mostrar los resultados:

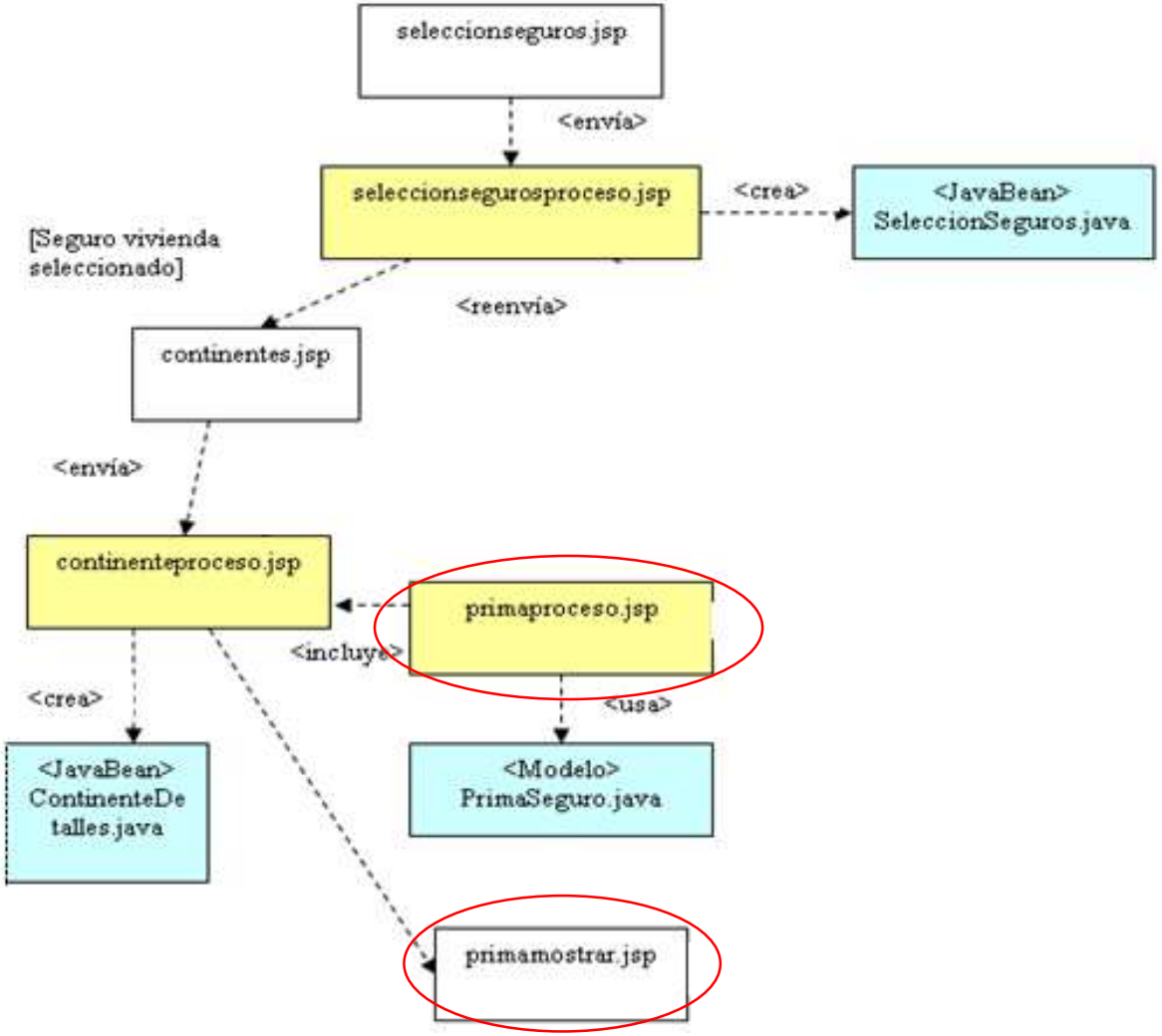
```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:useBean id="continente" class="beans.ContinenteDetalles" scope="session" />
<jsp:setProperty name="continente" property="*" />

<%
// seleccionar la siguiente página si también se requiere el seguro de contenidos
if(selseguros.getContenido()) {
%>
<jsp:forward page="contenidos.jsp" />
<%
}
else {
%>
<jsp:include page="primaproceso.jsp" />
<jsp:forward page="primamostrar.jsp" />
<%
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Gráficamente:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Vemos como la página ‘continentesproceso.jsp’ es la responsable únicamente de gestionar la redirección a una u otra página dentro del flujo de la aplicación.

- En cambio, la página ‘primaproceso.jsp’ tiene que encargarse tanto de la interacción con el modelo (formado por la clase PrimaSeguro.java) como de obtener la cuota del seguro.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- A continuación se muestra la página de formulario ‘contenidos.jsp’:

```
<?xml version="1.0"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
 <head>
  <link href="seguroscentrales.css" rel="stylesheet" type="text/css" />
  <title>SegurosCentrales - Detalles de las pertenencias</title>
 </head>
 <body>
 <!-- Fichero: contenidos.jsp -->
  <h1>Detalles de la poliza de pertenencias</h1>
  <h2>Por favor, suministre la informacion sobre las pertenencias que quiere asegurar en el
siguiente formulario. A continuacion pulse sobre <i>Enviar</i>
</h2>
  <form action="contenidosproceso.jsp" method="post">
   <table>
    <tr>
     <td><label for="coberturaAccidentes">Cobertura de daños por accidente (marcar
casilla)</label></td>
     <td><input type="checkbox" name="coberturaAccidentes" id="coberturaAccidentes" /></td>
    </tr>
    <tr>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

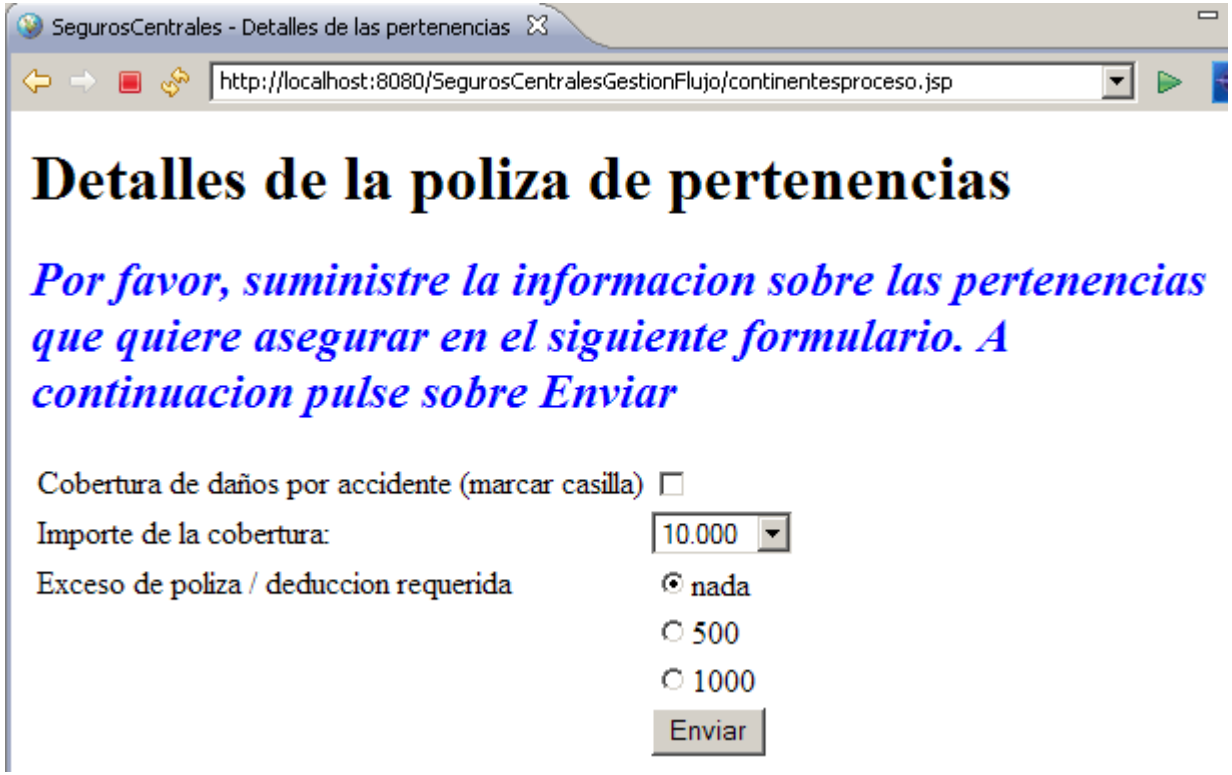
Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
<td><label for="importeCobertura">Importe de la cobertura:</label></td>
     <td>
      <select name="importeCobertura" size=1>
       <option value="10000">10.000</option>       <option value="20000">20.000</option>
       <option value="30000">30.000</option>       <option value="50000">50.000</option>
       <option value="100000">100.000</option>
      </select>
      </td> </tr>
     <tr>
      <td><label for="importeDeducible">Exceso de poliza / deduccion requerida</label></td>
      <td>
       <input type="radio" name="importeDeducible" value="0" checked="checked" />nada
      </td>     </tr>
     <tr>
      <td></td>
      <td>       <input type="radio" name="importeDeducible" value="500" />500      </td>
     </tr>
     <tr>
      <td></td>
      <td>       <input type="radio" name="importeDeducible" value="1000" />1000      </td>
     </tr>
     <tr>
      <td></td>
      <td>       <input type="submit" value="Enviar" />      </td>
     </tr>
    </table>
  </form>
 </body>
</html>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Aspecto del formulario ‘contenidos.jsp’:



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- El formulario anterior se envía a la página ‘contenidosproceso.jsp’.

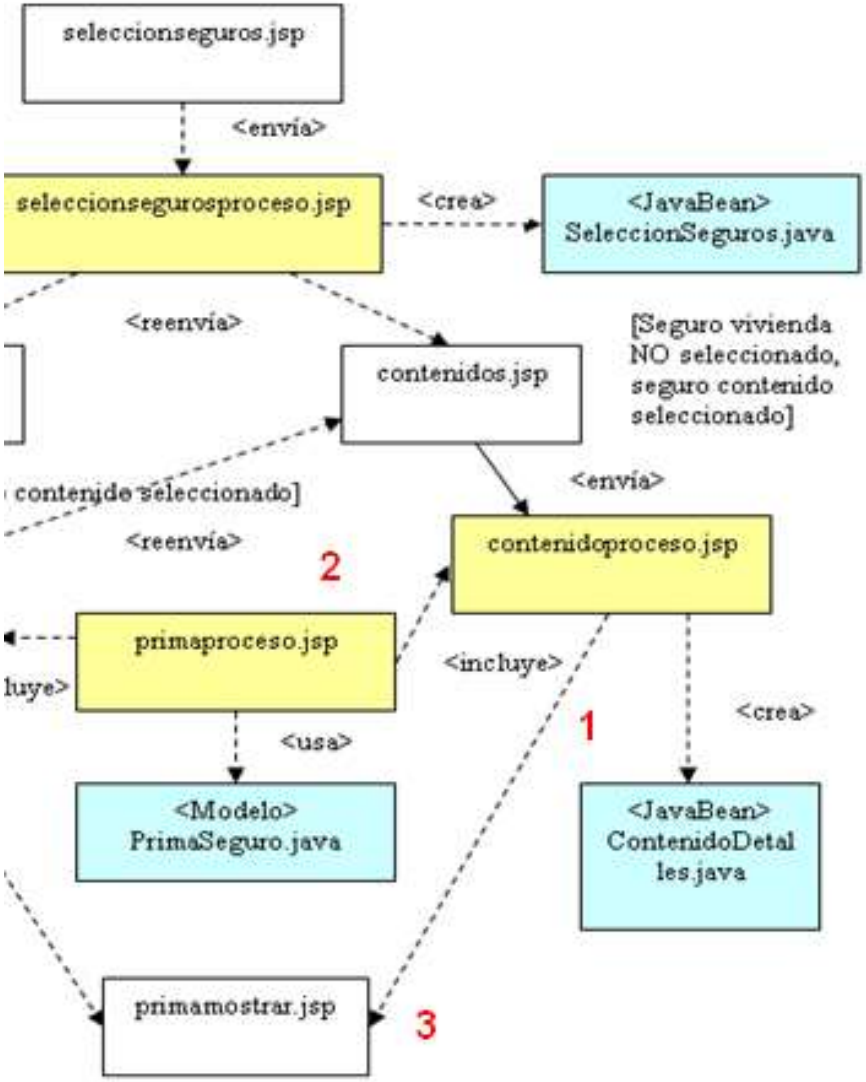
- Vemos como esta página es muy sencilla, tan sólo tiene que rellenar el bean con la información del formulario, incluir la página que calcula la prima y redireccionar el flujo de la aplicación a la página que muestra los resultados al usuario:

```
<jsp:useBean id="contenido" class="beans.ContenidoDetalles" scope="session" />
<jsp:setProperty name="contenido" property="*" />
<jsp:include page="primaproceso.jsp" />
<jsp:forward page="primamostrar.jsp" />
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- Gráficamente:

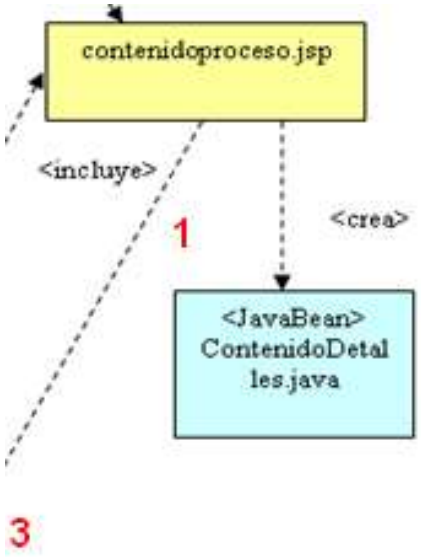


GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- El bean ContenidoDetalles tiene mucho en común con el bean ContinenteDetalles.

- Ambos tienen propiedades que coinciden con el formulario enviado desde el navegador, además de disponer de atributos para almacenar las cuotas de los presupuestos que se calcularán en un paso posterior.



GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

- El código de ContenidoDetalles.java:

```
package beans;

import java.text.NumberFormat;
import java.io.Serializable;

public class ContenidoDetalles implements Serializable {

  private static final long serialVersionUID = 1L;

  private boolean coberturaAccidentes;
  private int importeCobertura;
  private int importeDeducible;

  private double primaSeguro;

  public ContenidoDetalles() {
    super();
  }

  // Este método se llama getCoberturaAccidentes en vez de
  // isCoberturaAccidentes para ser compatible con JSP
  public boolean getCoberturaAccidentes() {
    return coberturaAccidentes;
  }
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
public void setCoberturaAccidentes(boolean coberturaAccidentes) {
  this.coberturaAccidentes = coberturaAccidentes;
}

public int getImporteCobertura() {
  return importeCobertura;
}

public void setImporteCobertura(int importeCobertura) {
  this.importeCobertura = importeCobertura;
}

public int getImporteDeducible() {
  return importeDeducible;
}

public void setImporteDeducible(int importeDeducible) {
  this.importeDeducible = importeDeducible;
}


public double getPrimaSeguro() {
  return primaSeguro;
}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Creación de JSP y JavaBean a partir del flujo de la aplicación Web

```
  public String getPrimaSeguroFormateada()
  {
    NumberFormat monedaFormato = NumberFormat.getCurrencyInstance();
    return monedaFormato.format(getPrimaSeguro());
  }

  public void setPrimaSeguro(double primaSeguro) {
    this.primaSeguro = primaSeguro;
  }

}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Hasta el momento las páginas JSP de nuestra aplicación simplemente recogen información del usuario y la almacenan en la sesión a través de JavaBean.

- Todavía no hemos utilizado ningún proceso de negocio que utilice estos datos, aunque evidentemente estos procesos son claves dentro de cualquier sistema informático.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- En nuestro caso los procesos de negocio son los relativos al cálculo de la prima del seguro en función de los datos que nos proporcione el usuario.

- La pregunta es ¿dónde hemos de colocar estos procesos de negocio?

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Debemos recordad que tenemos que separar lo más posible las distintas partes de una aplicación Web.

- Las páginas JSP que funcionan como controladores se encargan de atender las peticiones que hacen los clientes y de ejecutar los métodos de negocio.

- Las páginas JSP que funcionan como vistas se encargan de la presentación del contenido dinámico, pero no de los procesos que proporcionan dicho contenido.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Los JavaBean que hemos visto hasta el momento son DTO (Data Transfer Object), cuyo papel es encapsular los datos para enviarlos a través de los distintos componentes de la aplicación.

- Dentro de esta funcionalidad no se incluyen los procesos de negocio, aunque sí la validación y el correcto formateo de los datos.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Para implementar el modelo de negocio utilizaremos una sencilla clase que funcione como bean de proceso (aunque se podría utilizar EJB –Enterprise Java Bean).

- Un bean de proceso es un objeto Java que encapsula uno o más procesos de negocio que guardan cierta relación.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- En nuestro caso, necesitaremos un bean de proceso que pueda calcular la cuota para cada tipo de seguro.

- El bean de proceso se llamará PrimaSeguro.java y tendrá dos métodos:

```
public static double getPrimaContinente(ContinenteDetalles continenteDetalles)
public static double getPrimaContenido(ContenidoDetalles contenidoDetalles)
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

```
public static double getPrimaContinente(ContinenteDetalles continenteDetalles)
public static double getPrimaContenido(ContenidoDetalles contenidoDetalles)
```

- Estos métodos utilizarán objetos JavaBean como parámetros, que se procesarán para devolver un resultado numérico (la prima), sin necesidad de almacenar nada localmente.

- Por tanto, no se necesita crear instancia alguna de PrimaSeguro.java y estos métodos pueden ser estáticos.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Estos métodos estáticos serán invocados desde la JSP ‘primaproceso.jsp’, dado que sabemos que esa página es incluida por otras JSP (‘continentesproceso.jsp’ y/o ‘contenidosproceso.jsp’) que ya disponen de todos los datos de los clientes recabados de los formularios.

- ¿En qué consiste la implementación de la clase PrimaSeguro? Pues en una serie de operaciones matemáticas, que sin ser reales, se aproximan al cálculo que hacen las compañías de seguros.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- El código de ‘PrimaSeguro.java’:

```
package modelo;

import beans.ContinenteDetalles;
import beans.ContenidoDetalles;

public class PrimaSeguro {

  public static final int CUATROVIENTOS = 1;
  public static final int SEMIADOSADA = 2;
  public static final int ADOSADA = 3;
  public static final int PISO = 4;

  public static double getPrimaContinente(ContinenteDetalles continenteDetalles)
  {
  System.out.println("en PrimaSeguro::getPrimaContinente");
  double prima = 0.0;
  double primaBasica = continenteDetalles.getValorMercado() * .002;
  int continente = 0;
  String tipoPropiedad = continenteDetalles.getTipoPropiedad();

  if (tipoPropiedad.equals("piso")) {
    continente = PISO;
  }
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

```
if (tipoPropiedad.equals("semiadosada")) {
  continente = SEMIADOSADA;
}

if (tipoPropiedad.equals("adosada"))  {
  continente = ADOSADA;
}

if (tipoPropiedad.equals("aislada")) {
  continente = CUATROVIENTOS;
}

switch (continente) {
  case CUATROVIENTOS :
    prima= primaBasica* 1.1;
    break;
  case SEMIADOSADA:
    prima= primaBasica* 1.05;
    break;
  case ADOSADA:
    prima= primaBasica* 1;
    break;
  case PISO:
    prima= primaBasica* 0.95;
}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

```
prima= prima+ (continenteDetalles.getNumeroHabitaciones() * (prima/ 100));
String anyoCadena = continenteDetalles.getFechaConstruccion().substring(7, 10);
int anyo = Integer.parseInt(anyoCadena);

switch (anyo) {
  case 00 :
    prima+= prima* .04;
    break;
  case 01 :
    prima+= prima* .03;
    break;
  case 51 :
    prima+= prima* .02;
    break;
  case 71 :
    prima+= prima* .01;
 }

 if (continenteDetalles.getTipoConstruccion().equals("madera"))
 {
   prima+= (prima* 0.1);
 }

 System.out.println("Prima calculada " + prima);
  return prima;
}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

```
  public static double getPrimaContenido(ContenidoDetalles contenidoDetalles)
  {
    System.out.println("en PrimaSeguro::getPrimaContenido");
    boolean coberturaAccidentes = contenidoDetalles.getCoberturaAccidentes();
    int cobertura = contenidoDetalles.getImporteCobertura();
    int deducible = contenidoDetalles.getImporteDeducible();

    // Calcular la prima inicial

    double prima = cobertura * 0.004;

    if (coberturaAccidentes) {
      prima *= 1.25;
    }
    if(deducible>= 1000) {
      prima -= (prima * 0.2);
    }
    else {
     if(deducible>= 500) {
       prima -= (prima * 0.1);
     }
   }
   return prima ;
 }

}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Nos resta crear la JSP ‘primaproceso.jsp’, la cual interactúa con los distintos bean de la aplicación.

- Recupera los tres bean DTO de la sesión y los pasa al bean de proceso (PrimaSeguro).

- Finalmente graba las primas de los seguros recién calculadas en los beans correspondientes.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

```
<%@ page import="modelo.PrimaSeguro" %>
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:useBean id="continente" class="beans.ContinenteDetalles" scope="session" />
<jsp:useBean id="contenido" class="beans.ContenidoDetalles" scope="session" />

<%
// Obtener la prima para una poliza de continente (si se requiere)
if(selseguros.getContinente())
{
 double primaContinente = PrimaSeguro.getPrimaContinente(continente);

 // actualizar los detalles del continente con la prima
 continente.setPrimaSeguro(primaContinente);
}

//Obtener la prima para una poliza de contenido (si se requiere)
if(selseguros.getContenido())
{
 double primaContenido = PrimaSeguro.getPrimaContenido(contenido);

 //actualizar los detalles del contenido con la prima
 contenido.setPrimaSeguro(primaContenido);
}
%>
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

- Por último, tanto ‘continentesproceso.jsp’ como ‘contenidosproceso.jsp’ llevarán el flujo de la aplicación a su última página: ‘primamostrar.jsp’.

- ‘primamostrar.jsp’ es una vista que se encarga de recuperar los beans de la sesión para visualizar las primas calculadas mediante etiquetas jsp:getProperty.

- Finalmente invalida la sesión del usuario para liberar los recursos que se encuentren en el servidor, dado que ya no serán necesarios.

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

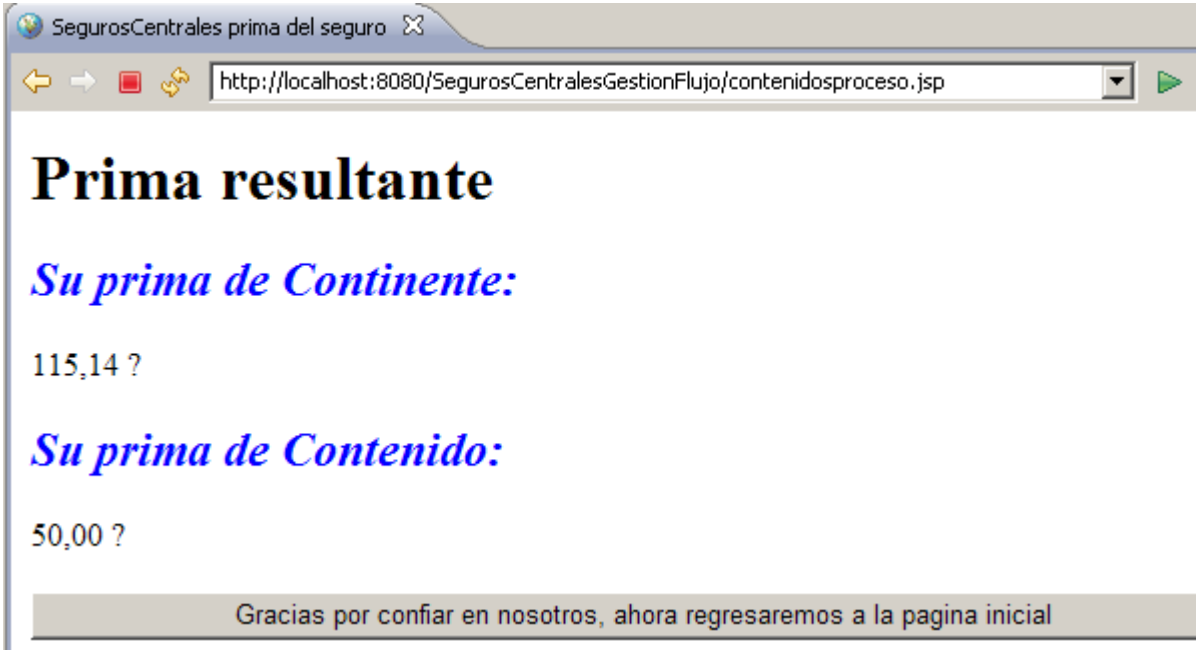
- Código de ‘primamostrar.jsp’:

```
<jsp:useBean id="selseguros" class="beans.SeleccionSeguros" scope="session" />
<jsp:useBean id="contenido" class="beans.ContenidoDetalles" scope="session" />
<jsp:useBean id="continente" class="beans.ContinenteDetalles" scope="session" />

<!DOCTYPE html PUBLIC
 "-//W3C//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
<head>
 <link href="seguroscentrales.css" rel="stylesheet" type="text/css" />
 <title>SegurosCentrales prima del seguro</title>
</head>

<body>
<h1>Prima resultante</h1>
<%
if(selseguros.getContinente())
{
%>
<h2>Su prima de Continente:</h2>
<jsp:getProperty name="continente" property="primaSeguroFormateada"/>
<%
}
```

GESTIÓN DE LA SESIÓN DE UNA APLICACIÓN WEB

Bean de proceso y el modelo de datos subyacente

```
if(selseguros.getContenido())
{
%>
<h2>Su prima de Contenido:</h2>
<jsp:getProperty name="contenido" property="primaSeguroFormateada"/>
<%
}
session.invalidate();
%>
<form action="bienvenido.jsp" method="get">
<input type="submit" value="Gracias por confiar en nosotros, ahora regresaremos a la
pagina inicial" />
</form>
</body>
</html>
```